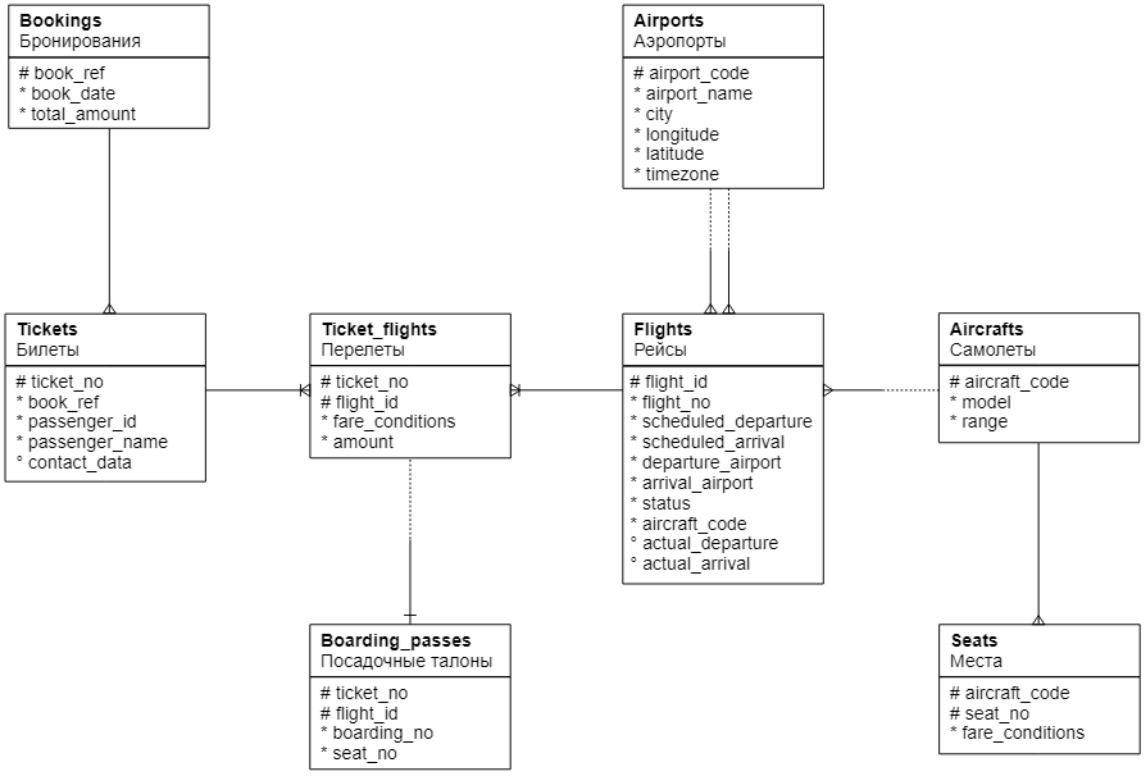

1 Рита FlightManagementPackage

Создание пакета:

- Создайте пакет "FlightManagementPackage"

Функции:

- Создайте в пакете "FlightManagementPackage" функцию **"IsFlightFull"**, которая принимает ID рейса и возвращает флаг (0 или 1). Функция должна проверять, заполнен ли рейс на максимальную вместимость.
- Создайте в пакете "FlightManagementPackage" функцию "**GetAvailableFlights**", которая принимает дату и пункт назначения, а затем возвращает информацию о доступных рейсах на указанную дату и направление.

Процедура с использованием функций и курсоров:

- В пакете "FlightManagementPackage" создайте процедуру "BookFlight", которая принимает параметры: ID пассажира, дату, пункт отправления и пункт назначения. Процедура должна:
- Проверять наличие пассажира в системе.
- Использовать функцию "GetAvailableFlights". для выборки информации о доступных рейсах на указанную дату и направление.

- Проверить максимальную загрузку рейса с использованием функции "IsFlightFull".
- Если найдены доступные рейсы и в них есть места, то необходимо сохранить эти данные в отдельную таблицу. Для этого используйте явные курсоры.
- Если все рейсы заполнены или нет доступных, а также если указанный пассажир не был найден в базе данных, то необходимо выбросить исключение с четким сообщением об ошибке.

```
CREATE OR REPLACE PACKAGE FlightManagementPackage AS  -- спецификация
    TYPE flightsList is table of (NUMBER(38,0));

  FUNCTION IsFlightFull (
      id_flight    NUMBER) RETURN number;
  FUNCTION GetAvailableFlights (
      DATE_F    DATE,
    CITY_b VARCHAR2) RETURN flightsList;
  PROCEDURE BookFlight (
      ID_PASSENGER    VARCHAR2,
    DATE_V    DATE,
    CITY_A VARCHAR2,
    CITY_B VARCHAR2);
  FUNCTION AirportsCity ( city_a    VARCHAR2) RETURN varchar2;

  passenger_not_found exception;
   flights_not_found exception;
   FlightFull exception;

END FlightManagementPackage;
/
CREATE OR REPLACE PACKAGE BODY FlightManagementPackage AS  -- тело

    CURSOR f1(CITY_A AVIA.AIRPORTS_DATA.CITY%TYPE) IS
        SELECT f.* FROM AVIA.FLIGHTS F, AVIA.AIRPORTS_DATA A
    WHERE CITY_A = SUBSTR(A.CITY, 1, INSTR(A.CITY, '/') - 1)
    AND AIRPORT_CODE = departure_AIRPORT;

    FUNCTION IsFlightFull (
        id_flight    NUMBER)
        RETURN number IS
        v_renge number;
        v_renge2 number;
    BEGIN
      SELECT COUNT(b.TICKET_NO) INTO V_RENGE
      FROM AVIA.ticket_flights B
```

```
    WHERE B.FLIGHT_ID = ID_FLIGHT;

        SELECT A.RANGE INTO v_renge2
        FROM AVIA.AIRCRAFTS_DATA A, AVIA.FLIGHTS F
        WHERE A.AIRCRAFT_CODE = F.AIRCRAFT_CODE
        AND id_flight = F.FLIGHT_ID;

        IF V_RENGE < v_renge2 THEN
      RETURN 1;
    ELSE
      RETURN 0;
    END IF;
  END IsFlightFull;

  FUNCTION GetAvailableFlights (
      DATE_F    DATE,
      CITY_b VARCHAR2)
    RETURN flightsList IS
    V_FLIGHTS flightsList := flightsList();
    BEGIN
        SELECT FLIGHT_ID bulk collect into V_FLIGHTS
        FROM AVIA.FLIGHTS F, AVIA.AIRPORTS_DATA A
        WHERE (City_b = SUBSTR(A.CITY, 1, INSTR(A.CITY, '/') - 1)) AND
A.AIRPORT_CODE = F.ARRIVAL_AIRPORT
        AND TRUNC(date_f) = TRUNC(F.SCHEDULED_DEPARTURE);

        RETURN V_FLIGHTS;
  END GetAvailableFlights;

  PROCEDURE BookFlight (
      ID_PASSENGER    VARCHAR2,
      DATE_V    DATE,
      CITY_A VARCHAR2,
      CITY_B VARCHAR2) IS

      v_flag1 number := 0;
      v_flag2 number := 0;
      V_passenger number := 0;
      V_count   number :=0;
      V_available_flights flightsList;
      flights_rec f1%ROWTYPE;
    BEGIN
    delete temp_table;

        SELECT count(PASSENGER_ID) INTO V_passenger FROM AVIA.TICKETS where
PASSENGER_ID = ID_PASSENGER;

    IF V_passenger = 0 THEN
```

```
      raise   passenger_not_found;
    END IF;

        V_available_flights := GetAvailableFlights(DATE_V, CITY_b);
        OPEN f1(CITY_A);
        LOOP
           FETCH f1 INTO flights_rec;
           EXIT WHEN f1%NOTFOUND;
           v_count :=0;
           for i in 1..V_available_flights.count loop
              select count(*) into v_count from avia.flights f where V_available_flights(i) =
flights_rec.flight_id and  flights_rec.flight_id = f.flight_id;
              if v_count = 1 then
                 exit;
              end if;
           end loop;
           IF V_count > 0 THEN
              v_flag1:= v_flag1 + 1;
              IF IsFlightFull(flights_rec.FLIGHT_ID) = 1 THEN
                 v_flag2:= v_flag2 + 1;
                 INSERT into temp_table VALUES(flights_rec.FLIGHT_ID, CITY_A,
flights_rec.departure_airport,  CITY_b, flights_rec.arrival_airport,
flights_rec.scheduled_departure);
              END IF;
           END IF;
        END LOOP;
        close f1;
        IF v_flag1 = 0 then
    raise   flights_not_found;
        END IF;

        IF v_flag2 = 0 then
    raise   FlightFull;
        END IF;


dbms_output.put_line('рейсы найдены успешно! проверьте таблицу temp_table (select *
from temp_table;)');
        exception
WHEN passenger_not_found THEN
              dbms_output.put_line('Пассажир не найден!');
           WHEN flights_not_found THEN
              dbms_output.put_line('рейсы не найдены!');
           WHEN FlightFull THEN
              dbms_output.put_line('мест нет!');
   END BookFlight;

  FUNCTION AirportsCity (
```

```
    city_a    VARCHAR2)
    RETURN varchar2 IS

      v_names VARCHAR2(4000); -- Максимальная длина строки
      BEGIN
         SELECT LISTAGG(SUBSTR(a.airport_name, 1, INSTR(a.airport_name, '/') - 1), ', ')
WITHIN GROUP (ORDER BY a.airport_name)
         INTO v_names
         FROM AVIA.AIRPORTS_DATA a
         WHERE SUBSTR(a.city, 1, INSTR(a.city, '/') - 1) = city_a;

         RETURN v_names;
   END AirportsCity;
END FlightManagementPackage;
```

2 Маша FlightReservationPackage

Создание пакета:

- Создайте пакет "FlightReservationPackage".

Функции:

- Создайте в пакете "FlightReservationPackage" функцию "IsFlightAvailable", которая принимает параметры ID рейса, класс билета и дату полета, и возвращает количество доступных билетов. Функция должна проверять доступность билетов выбранного класса на указанный рейс и дату.

- Создайте в пакете "FlightReservationPackage" функцию "GetPassengerReservations", которая принимает ID пассажира и возвращает информацию о его текущих бронированиях.

Процедура с использованием функций и курсоров:

- В пакете "FlightReservationPackage" создайте процедуру "MakeReservation", которая принимает параметры: ID пассажиров (от 1 до 10), ID рейса, класс билета и дату полета. Процедура должна:

- Проверять наличие пассажиров в системе.

- Для выборки информации о доступных билетах на указанный рейс и дату с использовать функцию "IsFlightAvailable".

- Проверять доступность выбранного класса билетов.

- Если билеты доступны, добавить новую запись в таблицу бронирования и в билеты. После завершения работы все добавленные записи необходимо удалить. За порчу исходной таблицы штраф (-1 балл за оставление лишних записей после себя, -2 балла за удаления чего-то кроме своих записей из исходных таблиц).

- Если нужное количество билетов не доступно или указанные пассажиры отсуствуют в базе данных выбросить исключение с четким сообщением об ошибке.

```
SELECT * FROM avia.flights;
SELECT * FROM avia.ticket_flights;
SELECT * FROM avia.tickets order by passenger_id;
SELECT * FROM AVIA.bookings;
SELECT * FROM avia.tickets where passenger_id='0000 000343';
SELECT * FROM avia.tickets;
SELECT * FROM avia.seats;
select * from airports;
select * from avia.ticket_flights where flight_id='336';
select * from avia.flights;
select * from avia.boarding_passes;
--1902 281414 336 business 12.09.17 10:40:00,000000000
--453 8926 645860 business 10.08.17 14:10:00,000000000
--id пассажира номер полета класс билета дата
--335 business,economy


CREATE OR REPLACE PACKAGE FlightReservationPackage AS
	FUNCTION IsFlightAvailable(fl_id IN NUMBER, fare_con IN VARCHAR2, flight_date
IN VARCHAR2) RETURN NUMBER;
	FUNCTION GetPassengerReservations(pas_id IN VARCHAR2) RETURN
VARCHAR2;
  PROCEDURE MakeReservation(pas_ids IN VARCHAR2, fl_id IN NUMBER, fare_con IN
VARCHAR2, flight_date IN VARCHAR2);
	FUNCTION check_passenger_ids (p_passenger_ids IN VARCHAR2) RETURN
VARCHAR2;
END FlightReservationPackage;


CREATE OR REPLACE PACKAGE BODY FlightReservationPackage AS
--Создайте в пакете FlightReversationPackage функцию IsFlightAvailable, которая
принимает параметры ID рейса класс билета и дату полета и возвращает количество
доступных билетов.
FUNCTION IsFlightAvailable(fl_id IN NUMBER, fare_con IN VARCHAR2, flight_date IN
VARCHAR2) RETURN NUMBER AS
	available_seats NUMBER;
   BEGIN
      SELECT COUNT(*) INTO available_seats
```

```
        FROM avia.flights f
        JOIN avia.ticket_flights tf ON tf.flight_id = f.flight_id
        WHERE (f.flight_id = fl_id
        AND TO_TIMESTAMP(f.scheduled_departure, 'DD.MM.YY HH24:Mi:SS,FF') =
TO_TIMESTAMP(flight_date, 'DD.MM.YY HH24:Mi:SS,FF')
        AND tf.fare_conditions = fare_con);
                    dbms_output.put_line( fare_con);
        RETURN available_seats;
    END IsFlightAvailable;
--Создайте в пакете FlightReversationPackage функцию GetPassengerReservations,
которая принимает ID пассажира и возвращает информацию о его текущих
бронированиях.
FUNCTION GetPassengerReservations(pas_id IN VARCHAR2) RETURN VARCHAR2 AS
        v_info VARCHAR2 (10000) := '';
        v_book_ref VARCHAR2 (50);
        v_book_date DATE;
        v_total_amount NUMBER;
        v_ticket_no VARCHAR2 (50);
        v_passenger_name VARCHAR2 (50);
        v_contact_data VARCHAR2 (50);
    BEGIN
        FOR rec IN (SELECT b.book_ref, b.book_date, b.total_amount, t.ticket_no,
t.passenger_name, t.contact_data
                    FROM AVIA.Bookings b
                    JOIN AVIA.Tickets t ON b.book_ref = t.book_ref
                    WHERE t.passenger_id = pas_id)
        LOOP
            v_book_ref := rec.book_ref;
            v_book_date := rec.book_date;
            v_total_amount := rec.total_amount;
            v_ticket_no := rec.ticket_no;
            v_passenger_name := rec.passenger_name;
            v_contact_data := rec.contact_data;
            v_info := v_info || 'Booking Reference: ' || v_book_ref || ', Booking Date: ' ||
v_book_date || ', Total Amount: ' || v_total_amount || ', Ticket No: ' || v_ticket_no || ',
Passenger Name: ' || v_passenger_name || ', Contact Data: ' || v_contact_data || CHR(10);
        END LOOP;
        RETURN v_info;
    END GetPassengerReservations;
--В пакете FlightReversationPackage создайте процедуру "MakeReservation", которая
принимает параметры ID пассажира (от 1 до 10), ID рейса, класс билета и дату полета.
--Процедура должна:
--Проверять наличие пассажиров в системе
--Для выборки инофрмации о доступных билетах на указанный рейс и дату и
использовать функцию IsFlightAvailable
--Проверять доступность выбранного класса билетов
--Если билеты доступны добавить новую запись в таблицу бронирования и в билеты
```

```
--Если нужное количество билетов не доступно или указанные пассажиры отсутствуют
в базе данных выбросить исключение с четким сообщением об ошибке
PROCEDURE MakeReservation(pas_ids IN VARCHAR2, fl_id IN NUMBER, fare_con IN
VARCHAR2, flight_date IN VARCHAR2) IS

                v_timer VARCHAR2(30) := TO_DATE(SYSDATE);
                v_passenger_count NUMBER;
                v_random VARCHAR2(10);
                v_current_pas_id VARCHAR2(20);

        BEGIN

      delete laba_BILETS;
      delete laba_BOOKINGS;
      delete laba_TICKETS;
      commit;


--Ошибка в случае отсутствия пассажира

      if ( (check_passenger_ids(pas_ids) LIKE '%НЕТ%') OR (LENGTH(fl_id) -
LENGTH(REPLACE(fl_id, ',', ''))+1 > IsFlightAvailable(fl_id, fare_con, flight_date)) ) then

        if ( (check_passenger_ids(pas_ids) LIKE '%НЕТ%')) then

                Insert into LOGS( ID_num, PAS_EXIST, COUNT_BILET, PROC_START,
PROC_END , FLAG, oshibka) values( seq_id.nextval, check_passenger_ids(pas_ids),
00000 , v_timer, TO_DATE(SYSDATE), 'НЕТ', 'Такого пользователя не существует!');

        end if;



        if ( (LENGTH(fl_id) - LENGTH(REPLACE(fl_id, ',', ''))+1 > IsFlightAvailable(fl_id,
fare_con, flight_date)) ) then

                Insert into LOGS( ID_num, PAS_EXIST, COUNT_BILET, PROC_START,
PROC_END , FLAG, oshibka) values( seq_id.nextval, check_passenger_ids(pas_ids),
00000 , v_timer, TO_DATE(SYSDATE), 'НЕТ', 'Недостаточно билетов!');

        end if;

      else

                        v_passenger_count := REGEXP_COUNT(pas_ids, ',') + 1;
            dbms_output.put_line(  v_passenger_count);
```

```
FOR oper_rec IN (SELECT tf.TICKET_NO, tf.FARE_CONDITIONS, tf.AMOUNT,
f.FLIGHT_ID, f.FLIGHT_NO, f.SCHEDULED_DEPARTURE, f.SCHEDULED_ARRIVAL,
f.DEPARTURE_AIRPORT, f.ARRIVAL_AIRPORT, f.STATUS, f.AIRCRAFT_CODE
        FROM avia.flights f
        JOIN avia.ticket_flights tf ON tf.flight_id = f.flight_id

        WHERE (f.flight_id = fl_id
        AND TO_TIMESTAMP(f.scheduled_departure, 'DD.MM.YY
HH24:Mi:SS,FF') = TO_TIMESTAMP(flight_date, 'DD.MM.YY HH24:Mi:SS,FF')
        AND tf.fare_conditions = fare_con)
        ) LOOP



            insert into laba_bilets values (oper_rec.TICKET_NO,
oper_rec.FARE_CONDITIONS, oper_rec.AMOUNT, oper_rec.FLIGHT_ID,
oper_rec.FLIGHT_NO, oper_rec.SCHEDULED_DEPARTURE,
oper_rec.SCHEDULED_ARRIVAL, oper_rec.DEPARTURE_AIRPORT,
oper_rec.ARRIVAL_AIRPORT, oper_rec.STATUS, oper_rec.AIRCRAFT_CODE);
            commit;

            if (v_passenger_count > 0) then

                    v_current_pas_id:=GetIDFromPosition(pas_ids,
v_passenger_count);
                    dbms_output.put_line(  v_current_pas_id);
                    v_random:=SUBSTR(DBMS_RANDOM.STRING('A', 6), 1,
6);
                    Insert into laba_tickets(ticket_no, book_ref, passenger_id,
passenger_name, contact_data) values( oper_rec.TICKET_NO, v_random,
v_current_pas_id,  getPassengerName(v_current_pas_id),
getContactDataByPassengerId(v_current_pas_id));
                    Insert into laba_bookings(book_ref, book_date,
TOTAL_AMOUNT) values(v_random, sysdate, oper_rec.AMOUNT);

                    v_passenger_count := v_passenger_count - 1;

            end if;


    END LOOP;


        Insert into LOGS( ID_num, PAS_EXIST, COUNT_BILET, PROC_START,
PROC_END , FLAG, OSHIBKA) values( seq_id.nextval, check_passenger_ids(pas_ids),
IsFlightAvailable(fl_id, fare_con, flight_date), v_timer, TO_DATE(SYSDATE), 'ДА', '-');
        Commit;
```

```
        end if;

END MakeReservation;
    FUNCTION check_passenger_ids(p_passenger_ids VARCHAR2)
    RETURN VARCHAR2
    IS
        v_result VARCHAR2(100) := 'ДА';
        v_missing_ids VARCHAR2(100) := '';
        v_count NUMBER;
    BEGIN
        FOR rec IN (
            SELECT REGEXP_SUBSTR(p_passenger_ids, '[^,]+', 1, level) AS passenger_id
            FROM dual
            CONNECT BY REGEXP_SUBSTR(p_passenger_ids, '[^,]+', 1, level) IS NOT NULL
        )
        LOOP
            BEGIN
                SELECT COUNT(*)
                INTO v_count
                FROM avia.Tickets
                WHERE passenger_id = rec.passenger_id;

                IF v_count = 0 THEN
                    v_result := 'НЕТ';
                    v_missing_ids := v_missing_ids || ',' || rec.passenger_id;
                END IF;
            END;
        END LOOP;

        IF v_result = 'НЕТ' THEN
            v_result := v_result || ' (Пользователь с этим id не найден: ' ||
SUBSTR(v_missing_ids, 2) || ')';
        END IF;

        RETURN v_result;
    END;

END FlightReservationPackage;



--Таблицы для выгрузки операций
create table laba_BILETS as
(SELECT tf.TICKET_NO, tf.FARE_CONDITIONS, tf.AMOUNT, f.FLIGHT_ID, f.FLIGHT_NO,
f.SCHEDULED_DEPARTURE, f.SCHEDULED_ARRIVAL, f.DEPARTURE_AIRPORT,
f.ARRIVAL_AIRPORT, f.STATUS, f.AIRCRAFT_CODE FROM AVIA.flights f
        JOIN AVIA.ticket_flights tf ON tf.flight_id = f.flight_id where rownum<2);
```

```
COMMIT;
CREATE TABLE laba_TICKETS AS
(SELECT * FROM AVIA.tickets where rownum<2);
COMMIT;
CREATE TABLE laba_BOOKINGS AS
(SELECT * FROM AVIA.bookings where rownum<2);
COMMIT;
--Таблица Логов
create table LOGS
  (
    ID_num number primary key,
    PAS_EXIST VARCHAR2(90) not null,
    COUNT_BILET NUMBER not null,

    PROC_START date,
    PROC_END date,

    FLAG VARCHAR2 (30),
    OSHIBKA VARCHAR2 (1000)
  );
  drop table LOGS;


--Последовательность
CREATE SEQUENCE SEQ_ID
START WITH 1
INCREMENT BY 1
NOMAXVALUE;

drop SEQUENCE SEQ_ID;

--Дроп таблиц
DROP TABLE laba_BILETS;
DROP TABLE laba_TICKETS;
DROP TABLE laba_BOOKINGS;
DROP TABLE LOGS;



--Select таблиц
SELECT * FROM laba_BILETS;
SELECT * FROM laba_TICKETS;
SELECT * FROM laba_BOOKINGS;
SELECT * FROM LOGS;
```

```
--Запросы для отладки
Begin
 FlightReservationPackage.MakeReservation('4631 903827' ,1567, 'Economy', '16.07.17
11:30:00,000000000');
end;
--0796 697225 1567 business 03.08.17 10:40:00,000000000
--  1567, 'Business', '16.07.17 11:30:00,000000000'
Begin
 FlightReservationPackage.MakeReservation('0000 000343', 1567, 'Business', '16.07.17
11:30:00,000000000');
end;
-------------------453 8926 645860 business 10.08.17 14:10:00,000000000
Begin
 FlightReservationPackage.MakeReservation('8926 645860', 453, 'business', '10.08.17
14:10:00,000000000');
end;
------------------
Begin
 FlightReservationPackage.MakeReservation('0000 000343', 335, 'Business', '16.07.17
11:30:00,000000000');
end;

Begin
 FlightReservationPackage.MakeReservation('7270 339448,0000 000343' ,1567, 'Economy',
'16.07.17 11:30:00,000000000');
end;


Begin
 FlightReservationPackage.MakeReservation('4903827' ,1567, 'Economy', '16.07.17
11:30:00,000000000');
end;

--1902 281414 336 business 12.09.17 10:40:00,000000000
Begin
 FlightReservationPackage.MakeReservation('1902 281414' ,336, 'business', '12.09.17
10:40:00,000000000');
end;

Begin
 FlightReservationPackage.MakeReservation('4631 903827,7270 339448' ,1567, 'Economy',
'16.07.17 11:30:00,000000000');
end;


Begin
 FlightReservationPackage.MakeReservation('4903827' ,1567, 'Economy', '16.07.17
11:30:00,000000000');
```

```
end;


Begin
 FlightReservationPackage.MakeReservation('4631 903827' ,1567, 'Econom', '16.07.17
11:30:00,000000000');
end;
--Дополнительные функции
CREATE OR REPLACE FUNCTION getPassengerName(passenger_id IN VARCHAR2)
RETURN VARCHAR2 AS
   v_passenger_name VARCHAR2(100);
BEGIN
   SELECT passenger_name INTO v_passenger_name FROM avia.Tickets WHERE
passenger_id = getPassengerName.passenger_id;
         RETURN v_passenger_name;
END;
CREATE OR REPLACE FUNCTION getContactDataByPassengerId(v_passenger_id IN
varchar2) RETURN varchar2 AS
         contact_data_value varchar2(100);
BEGIN
   SELECT contact_data INTO contact_data_value FROM avia.Tickets WHERE
v_passenger_id = passenger_id;
         RETURN contact_data_value;
END;

CREATE OR REPLACE FUNCTION GetIDFromPosition(input_string IN VARCHAR2,
position IN NUMBER) RETURN VARCHAR2 AS
   id_list VARCHAR2(255);
   id VARCHAR2(50);
BEGIN
   id_list := REPLACE(input_string, ',', ' ');
   id := TRIM(REGEXP_SUBSTR(id_list, '\d+ \d+', 1, position));
   RETURN id;
END;



--Проверка доп функций
SELECT GetIDFromPosition('4631 903827,4631 111111', 2) FROM dual;
SELECT SUBSTR(DBMS_RANDOM.STRING('A', 6), 1, 6) FROM dual;
SELECT getPassengerName('4631 903827') FROM dual;
SELECT getContactDataByPassengerId('4631 903827') FROM dual;

--Проверки
--Проверка первой функциии
SELECT f.scheduled_departure, tf.fare_conditions, f.flight_id,
FlightReservationPackage.IsFlightAvailable(  1567, 'Business', '16.07.17
```

```
11:30:00,000000000') FROM avia.flights f  JOIN avia.ticket_flights tf ON tf.flight_id =
f.flight_id  WHERE (f.flight_id = 1567
      AND TO_TIMESTAMP(f.scheduled_departure, 'DD.MM.YY HH24:Mi:SS,FF') =
TO_TIMESTAMP( '16.07.17 11:30:00,000000000', 'DD.MM.YY HH24:Mi:SS,FF')       AND
tf.fare_conditions = 'Business');
--Проверка второй функции
SELECT t.passenger_id, FlightReservationPackage.GetPassengerReservations('0000
000343')
            FROM AVIA.Bookings b
            JOIN AVIA.Tickets t ON b.book_ref = t.book_ref
            WHERE t.passenger_id = '0000 000343';


--Проверка третьей функции

select FlightReservationPackage.check_passenger_ids('0796 697225,123,12111') from dual;

select  passenger_id, FlightReservationPackage.check_passenger_ids(passenger_id)  from
avia.tickets;


SET SERVEROUTPUT ON;
```

3 Макс FlightReservationPackage

Создание пакета:

- Создайте пакет "FlightReservationPackage".

Функции:

- Создайте в пакете "FlightReservationPackage" функцию "IsFlightAvailable", которая принимает параметры ID рейса, класс билета и дату полета, и возвращает количество доступных билетов. Функция должна проверять доступность билетов выбранного класса на указанный рейс и дату.

- Создайте в пакете "FlightReservationPackage" функцию "GetPassengerReservations", которая принимает ID пассажира и возвращает информацию о его текущих бронированиях.

Процедура с использованием функций и курсоров:

- В пакете "FlightReservationPackage" создайте процедуру "MakeReservation", которая принимает параметры: ID пассажиров (от 1 до 10), ID рейса, класс билета и дату полета. Процедура должна:

- Проверять наличие пассажиров в системе.

- Для выборки информации о доступных билетах на указанный рейс и дату с использовать функцию "IsFlightAvailable".

- Проверять доступность выбранного класса билетов.

- Если билеты доступны, добавить новую запись в таблицу бронирования и в билеты. После завершения работы все добавленные записи необходимо удалить. За порчу исходной таблицы штраф (-1 балл за оставление лишних записей после себя, -2 балла за удаления чего-то кроме своих записей из исходных таблиц).

- Если нужное количество билетов не доступно или указанные пассажиры отсуствуют в базе данных выбросить исключение с четким сообщением об ошибке.

```
CREATE OR REPLACE PACKAGE BookingManagementPackage IS

  FUNCTION GetAvailableFlights RETURN SYS_REFCURSOR;

  FUNCTION GetFlightDetails(flightId IN NUMBER) RETURN avia.flights%ROWTYPE;

  PROCEDURE CancelBooking(booking_id IN VARCHAR2);

 FUNCTION GetFlightCityArrival(flight_id IN NUMBER) RETURN VARCHAR2;

END BookingManagementPackage;


drop  TABLE intermediate_table_tickets
drop TABLE intermediate_table_bookings
drop table testflights
drop  TABLE backup_bookings
drop PACKAGE BookingManagementPackage

CREATE OR REPLACE FUNCTION GetAvailableFlights(p_status VARCHAR2)
RETURN SYS_REFCURSOR
IS
  v_cursor SYS_REFCURSOR;
BEGIN
  OPEN v_cursor FOR
    SELECT f.flight_id, f.status
    FROM avia.flights f
    WHERE f.status = p_status;

  RETURN v_cursor;
```

```
END GetAvailableFlights;

DECLARE
  v_flight_id avia.flights.flight_id%TYPE;
  v_status avia.flights.status%TYPE;
  v_cursor SYS_REFCURSOR;
BEGIN
  v_cursor := GetAvailableFlights('Scheduled'); -- Вызов функции с параметром

  LOOP
    FETCH v_cursor INTO v_flight_id, v_status;
    EXIT WHEN v_cursor%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE('Flight ID: ' || v_flight_id || '  Status: ' || v_status); -- Вывод
идентификатора и статуса рейса
  END LOOP;

  CLOSE v_cursor;
END;
SET SERVEROUTPUT ON;

 CREATE OR REPLACE function GetFlightCityArrival(flight_id in NUMBER)
    return varchar2
    is
    v_result varchar2(1000);
    flight_info avia.flights%ROWTYPE;
    begin
      select *
      into flight_info
      from avia.flights
      where flight_id=GetFlightCityArrival.flight_id;
      select city into v_result
      from avia.airports_data
      where airport_code=flight_info.departure_airport;
      return v_result;

  end GetFlightCityArrival;

DECLARE
  v_flight_id NUMBER := 999999999;
  v_city_arrival VARCHAR2(1000);

BEGIN

  v_city_arrival := GetFlightCityArrival(v_flight_id);

  DBMS_OUTPUT.PUT_LINE('Город прибытия для рейса: '  || v_city_arrival);
EXCEPTION
  WHEN OTHERS THEN
```

```
  DBMS_OUTPUT.PUT_LINE('Такого самолёта не существует');
END;
SET SERVEROUTPUT ON;
```

```
CREATE OR REPLACE PACKAGE BODY BookingManagementPackage IS

 FUNCTION GetAvailableFlights
 RETURN sys_refcursor
 is
  v_cursor sys_refcursor;
 begin
  open v_cursor for
   select distinct f.flight_id
   from avia.flights f
   join AVIA.aircrafts_data ac on (f.aircraft_code = ac.aircraft_code)
   join avia.seats s on (ac.aircraft_code = s.aircraft_code)
   where (select count(*) from avia.ticket_flights tf where tf.flight_id = f.flight_id) <
         (select count(*) from avia.seats s where s.aircraft_code = ac.aircraft_code)
         and f.status != 'Cancelled'
         and f.status != 'On Time' and
         f.flight_id=335;
  --close v_cursor;
  return v_cursor;
 end GetAvailableFlights;

 function GetFlightDetails (flightId in NUMBER)
  return avia.flights%ROWTYPE
  is
  v_result avia.flights%ROWTYPE;
  begin
  select * into v_result
  from avia.flights f where f.flight_id=flightId;
  return v_result;
 end GetFlightDetails;

 PROCEDURE CancelBooking (booking_id IN varchar2)
 AS
   book_ref varchar2(6);
   flight_id number;
   CURSOR booking_cursor IS
      SELECT book_ref
```

```
        FROM backup_bookings
        WHERE book_ref = CancelBooking.booking_id;
    CURSOR flight_id_cursor IS
        SELECT f.flight_id
        FROM avia.flights f
        join avia.ticket_flights tf on (f.flight_id=tf.flight_id)
        join avia.tickets t on (tf.ticket_no=t.ticket_no)
        join avia.bookings b on (t.book_ref=b.book_ref)
        where b.book_ref=CancelBooking.booking_id;
  BEGIN
  -- очищаем старую информацию об отмененных бронированиях из таблиц
  delete intermediate_table_bookings;
  delete intermediate_table_tickets;
    OPEN booking_cursor;
    FETCH booking_cursor INTO book_ref;
    OPEN flight_id_cursor;
    FETCH flight_id_cursor into flight_id;

    IF booking_cursor%FOUND
    THEN
        if flight_id_cursor%NOTFOUND
          then RAISE_APPLICATION_ERROR(-20001, 'Бронь с таким ID не найдена');
          return;
        end if;
        --сохраняем запись о бронировании
        INSERT INTO intermediate_table_bookings (book_ref, book_date, total_amount)
        SELECT book_ref, book_date, total_amount
        FROM avia.bookings
        WHERE book_ref = CancelBooking.booking_id;

        --сохраняем запись о билетах
        INSERT INTO intermediate_table_tickets (ticket_no, book_ref, passenger_id,
passenger_name, contact_data)
        SELECT ticket_no, book_ref, passenger_id, passenger_name, contact_data
        FROM avia.tickets
        WHERE book_ref = CancelBooking.booking_id;

        --удаление записи из bookings
        DELETE FROM backup_bookings
        WHERE book_ref = CancelBooking.booking_id;

        --обратная вставка в bookings
       -- INSERT INTO backup_bookings bb (bb.book_ref, bb.book_date, bb.total_amount)
        --SELECT b.book_ref, b.book_date, b.total_amount FROM avia.bookings b
        --WHERE b.book_ref = CancelBooking.booking_id;

        DBMS_OUTPUT.PUT_LINE('Бронь успешно отменена.');
    ELSE
```

```
        RAISE_APPLICATION_ERROR(-20001, 'Бронь с указанным ID не существует.');
    END IF;
    CLOSE booking_cursor;
  EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20001, 'Бронь с указанным ID не существует.');
  END CancelBooking;

 function GetFlightCityArrival(flight_id in NUMBER)
   return varchar2
   is
   v_result varchar2(1000);
   flight_info avia.flights%ROWTYPE;
   begin
     select *
     into flight_info
     from avia.flights
     where flight_id=GetFlightCityArrival.flight_id;
     select city into v_result
     from avia.airports_data
     where airport_code=flight_info.departure_airport;
     return v_result;
     EXCEPTION
   WHEN NO_DATA_FOUND THEN
     RETURN 'Самолета не существует';
  end GetFlightCityArrival;


END BookingManagementPackage;

 SELECT book_ref
        FROM avia.bookings
        WHERE book_ref = '001F1D';


declare
  v_cursor sys_refcursor;
  v_flight_info avia.flights%ROWTYPE;
  v_flight number;
  v_flight_city varchar2(32767);
begin
  v_cursor := BookingManagementPackage.GetAvailableFlights();
  v_flight_info := BookingManagementPackage.GetFlightDetails(1);
  v_flight_city := BookingManagementPackage.GetFlightCityArrival(1);
  BookingManagementPackage.CancelBooking('0045DB');
loop
  fetch v_cursor into v_flight;
  insert into testflights values (v_flight);
```

```
  exit when v_cursor%NOTFOUND;
  end loop;
  DBMS_OUTPUT.PUT_LINE(v_flight_info.flight_id);
  DBMS_OUTPUT.PUT_LINE(v_flight_city);
  end;
  SET SERVEROUTPUT ON;

 select v_flight_info from avia.flights where flight_id=335;


  delete intermediate_table_bookings;
  delete intermediate_table_tickets;
select * from intermediate_table_tickets;
select * from intermediate_table_bookings;
SELECT * FROM backup_bookings;

CREATE  TABLE backup_bookings AS
    SELECT * FROM avia.bookings;


CREATE  TABLE intermediate_table_tickets (
    ticket_no varchar2(100),
    book_ref varchar2(100),
    passenger_id varchar2(100),
    passenger_name varchar2(100),
    contact_data varchar2(100)
);



CREATE  TABLE intermediate_table_bookings (
    book_ref varchar2(100),
    book_date date,
    total_amount number
);


create table testflights as select flight_id from avia.flights;
select * from testflights;

SELECT f.flight_id FROM avia.flights f
join avia.ticket_flights tf on (f.flight_id=tf.flight_id)
join avia.tickets t on (tf.ticket_no=t.ticket_no)
join avia.bookings b on (t.book_ref=b.book_ref)
where b.book_ref='31F31A';
```

4 Катя FlightReservationPackage

Создание пакета:

- Создайте пакет "FlightReservationPackage".

Функции:

- Создайте в пакете "FlightReservationPackage" функцию "IsFlightAvailable", которая принимает параметры ID рейса, класс билета и дату полета, и возвращает количество доступных билетов. Функция должна проверять доступность билетов выбранного класса на указанный рейс и дату.

- Создайте в пакете "FlightReservationPackage" функцию "GetPassengerReservations", которая принимает ID пассажира и возвращает информацию о его текущих бронированиях.

Процедура с использованием функций и курсоров:

- В пакете "FlightReservationPackage" создайте процедуру "MakeReservation", которая принимает параметры: ID пассажиров (от 1 до 10), ID рейса, класс билета и дату полета. Процедура должна:

- Проверять наличие пассажиров в системе.

- Для выборки информации о доступных билетах на указанный рейс и дату с использовать функцию "IsFlightAvailable".

- Проверять доступность выбранного класса билетов.

- Если билеты доступны, добавить новую запись в таблицу бронирования и в билеты. После завершения работы все добавленные записи необходимо удалить. За порчу исходной таблицы штраф (-1 балл за оставление лишних записей после себя, -2 балла за удаления чего-то кроме своих записей из исходных таблиц).

- Если нужное количество билетов не доступно или указанные пассажиры отсуствуют в базе данных выбросить исключение с четким сообщением об ошибке.

--Проверка третьей функции

```
select FlightReservationPackage.check_passenger_ids('0796 697225,123,12111') from dual;

select  passenger_id, FlightReservationPackage.check_passenger_ids(passenger_id)  from
avia.tickets;
```

--Таблицы для выгрузки операций

```
create table laba_BILETS as
```

```
(SELECT tf.TICKET_NO, tf.FARE_CONDITIONS, tf.AMOUNT, f.FLIGHT_ID, f.FLIGHT_NO,
f.SCHEDULED_DEPARTURE, f.SCHEDULED_ARRIVAL, f.DEPARTURE_AIRPORT,
f.ARRIVAL_AIRPORT, f.STATUS, f.AIRCRAFT_CODE FROM AVIA.flights f
        JOIN AVIA.ticket_flights tf ON tf.flight_id = f.flight_id where rownum<2);
COMMIT;
CREATE TABLE laba_TICKETS AS
(SELECT * FROM AVIA.tickets where rownum<2);
COMMIT;
CREATE TABLE laba_BOOKINGS AS
(SELECT * FROM AVIA.bookings where rownum<2);
COMMIT;
--Таблица Логов
create table LOGS
  (
    ID_num number primary key,
    PAS_EXIST VARCHAR2(90) not null,
    COUNT_BILET NUMBER not null,

    PROC_START date,
    PROC_END date,

    FLAG VARCHAR2 (30),
    OSHIBKA VARCHAR2 (1000)
  );
  drop table LOGS;


--Последовательность
CREATE SEQUENCE SEQ_ID
START WITH 1
INCREMENT BY 1
NOMAXVALUE;

drop SEQUENCE SEQ_ID;

--Дроп таблиц
DROP TABLE laba_BILETS;
DROP TABLE laba_TICKETS;
DROP TABLE laba_BOOKINGS;
DROP TABLE LOGS;


CREATE OR REPLACE PACKAGE FlightReservationPackage AS
        FUNCTION IsFlightAvailable(fl_id IN NUMBER, fare_con IN VARCHAR2, flight_date
IN VARCHAR2) RETURN NUMBER;
        FUNCTION GetPassengerReservations(pas_id IN VARCHAR2) RETURN
VARCHAR2;
```

```
  PROCEDURE MakeReservation(pas_ids IN VARCHAR2, fl_id IN NUMBER, fare_con IN
VARCHAR2, flight_date IN VARCHAR2);
        FUNCTION check_passenger_ids (p_passenger_ids IN VARCHAR2) RETURN
VARCHAR2;
END FlightReservationPackage;


CREATE OR REPLACE PACKAGE BODY FlightReservationPackage AS

FUNCTION IsFlightAvailable(fl_id IN NUMBER, fare_con IN VARCHAR2, flight_date IN
VARCHAR2) RETURN NUMBER AS
        available_seats NUMBER;
   BEGIN
      SELECT COUNT(*) INTO available_seats
      FROM avia.flights f
      JOIN avia.ticket_flights tf ON tf.flight_id = f.flight_id
      WHERE (f.flight_id = fl_id
      AND TO_TIMESTAMP(f.scheduled_departure, 'DD.MM.YY HH24:Mi:SS,FF') =
TO_TIMESTAMP(flight_date, 'DD.MM.YY HH24:Mi:SS,FF')
      AND tf.fare_conditions = fare_con);
                dbms_output.put_line( fare_con);
      RETURN available_seats;
   END IsFlightAvailable;

FUNCTION GetPassengerReservations(pas_id IN VARCHAR2) RETURN VARCHAR2 AS
      v_info VARCHAR2 (10000) := '';
      v_book_ref VARCHAR2 (50);
      v_book_date DATE;
      v_total_amount NUMBER;
      v_ticket_no VARCHAR2 (50);
      v_passenger_name VARCHAR2 (50);
      v_contact_data VARCHAR2 (50);
   BEGIN
      FOR rec IN (SELECT b.book_ref, b.book_date, b.total_amount, t.ticket_no,
t.passenger_name, t.contact_data
               FROM AVIA.Bookings b
               JOIN AVIA.Tickets t ON b.book_ref = t.book_ref
               WHERE t.passenger_id = pas_id)
      LOOP
         v_book_ref := rec.book_ref;
         v_book_date := rec.book_date;
         v_total_amount := rec.total_amount;
         v_ticket_no := rec.ticket_no;
         v_passenger_name := rec.passenger_name;
         v_contact_data := rec.contact_data;
         v_info := v_info || 'Booking Reference: ' || v_book_ref || ', Booking Date: ' ||
v_book_date || ', Total Amount: ' || v_total_amount || ', Ticket No: ' || v_ticket_no || ',
Passenger Name: ' || v_passenger_name || ', Contact Data: ' || v_contact_data || CHR(10);
```

```
    END LOOP;
    RETURN v_info;
  END GetPassengerReservations;

PROCEDURE MakeReservation(pas_ids IN VARCHAR2, fl_id IN NUMBER, fare_con IN
VARCHAR2, flight_date IN VARCHAR2) IS

                v_timer VARCHAR2(30) := TO_DATE(SYSDATE);
                v_passenger_count NUMBER;
                v_random VARCHAR2(10);
                v_current_pas_id VARCHAR2(20);

        BEGIN

      delete laba_BILETS;
      delete laba_BOOKINGS;
      delete laba_TICKETS;
      commit;

--Ошибка в случае отсутствия пассажира

      if ( (check_passenger_ids(pas_ids) LIKE '%HET%') OR (LENGTH(fl_id) -
LENGTH(REPLACE(fl_id, ',', ''))+1 > IsFlightAvailable(fl_id, fare_con, flight_date)) ) then

        if ( (check_passenger_ids(pas_ids) LIKE '%HET%')) then

                Insert into LOGS( ID_num, PAS_EXIST, COUNT_BILET, PROC_START,
PROC_END , FLAG, oshibka) values( seq_id.nextval, check_passenger_ids(pas_ids),
00000 , v_timer, TO_DATE(SYSDATE), 'HET', 'Такого пользователя не существует!');
                  end if;


        if ( (LENGTH(fl_id) - LENGTH(REPLACE(fl_id, ',', ''))+1 > IsFlightAvailable(fl_id,
fare_con, flight_date)) ) then

                Insert into LOGS( ID_num, PAS_EXIST, COUNT_BILET, PROC_START,
PROC_END , FLAG, oshibka) values( seq_id.nextval, check_passenger_ids(pas_ids),
00000 , v_timer, TO_DATE(SYSDATE), 'HET', 'Недостаточно билетов!');

        end if;

      else

                  v_passenger_count := REGEXP_COUNT(pas_ids, ',') + 1;
            dbms_output.put_line(  v_passenger_count);
```

```
            FOR oper_rec IN (SELECT tf.TICKET_NO, tf.FARE_CONDITIONS, tf.AMOUNT,
f.FLIGHT_ID, f.FLIGHT_NO, f.SCHEDULED_DEPARTURE, f.SCHEDULED_ARRIVAL,
f.DEPARTURE_AIRPORT, f.ARRIVAL_AIRPORT, f.STATUS, f.AIRCRAFT_CODE
                FROM avia.flights f
                JOIN avia.ticket_flights tf ON tf.flight_id = f.flight_id

                WHERE (f.flight_id = fl_id
                AND TO_TIMESTAMP(f.scheduled_departure, 'DD.MM.YY
HH24:Mi:SS,FF') = TO_TIMESTAMP(flight_date, 'DD.MM.YY HH24:Mi:SS,FF')
                AND tf.fare_conditions = fare_con)
                ) LOOP

                  insert into laba_bilets values (oper_rec.TICKET_NO,
oper_rec.FARE_CONDITIONS, oper_rec.AMOUNT, oper_rec.FLIGHT_ID,
oper_rec.FLIGHT_NO, oper_rec.SCHEDULED_DEPARTURE,
oper_rec.SCHEDULED_ARRIVAL, oper_rec.DEPARTURE_AIRPORT,
oper_rec.ARRIVAL_AIRPORT, oper_rec.STATUS, oper_rec.AIRCRAFT_CODE);
                  commit;
                  if (v_passenger_count > 0) then

                        v_current_pas_id:=GetIDFromPosition(pas_ids,
v_passenger_count);
                        dbms_output.put_line(  v_current_pas_id);
                        v_random:=SUBSTR(DBMS_RANDOM.STRING('A', 6), 1,
6);
                        Insert into laba_tickets(ticket_no, book_ref, passenger_id,
passenger_name, contact_data) values( oper_rec.TICKET_NO, v_random,
v_current_pas_id,  getPassengerName(v_current_pas_id),
getContactDataByPassengerId(v_current_pas_id));
                        Insert into laba_bookings(book_ref, book_date,
TOTAL_AMOUNT) values(v_random, sysdate, oper_rec.AMOUNT);
                              v_passenger_count := v_passenger_count - 1;
                                    end if;

                  END LOOP;

            Insert into LOGS( ID_num, PAS_EXIST, COUNT_BILET, PROC_START,
PROC_END , FLAG, OSHIBKA) values( seq_id.nextval, check_passenger_ids(pas_ids),
IsFlightAvailable(fl_id, fare_con, flight_date), v_timer, TO_DATE(SYSDATE), 'ДА', '-');
            Commit;

      end if;

END MakeReservation;
   FUNCTION check_passenger_ids(p_passenger_ids VARCHAR2)
   RETURN VARCHAR2
   IS
      v_result VARCHAR2(100) := 'ДА';
```

```
      v_missing_ids VARCHAR2(100) := '';
      v_count NUMBER;
   BEGIN
      FOR rec IN (
         SELECT REGEXP_SUBSTR(p_passenger_ids, '[^,]+', 1, level) AS passenger_id
         FROM dual
         CONNECT BY REGEXP_SUBSTR(p_passenger_ids, '[^,]+', 1, level) IS NOT NULL
      )
      LOOP
         BEGIN
            SELECT COUNT(*)
            INTO v_count
            FROM avia.Tickets
            WHERE passenger_id = rec.passenger_id;

            IF v_count = 0 THEN
               v_result := 'HET';
               v_missing_ids := v_missing_ids || ',' || rec.passenger_id;
            END IF;
         END;
      END LOOP;

      IF v_result = 'HET' THEN
         v_result := v_result || ' (Пользователь с этим id не найден: ' ||
SUBSTR(v_missing_ids, 2) || ')';
      END IF;

      RETURN v_result;
   END;

END FlightReservationPackage;

--Select таблиц
SELECT * FROM laba_BILETS;
SELECT * FROM laba_TICKETS;
SELECT * FROM laba_BOOKINGS;


SELECT * FROM LOGS;


--Проверки
--Проверка первой функциии
SELECT DISTINCT  f.scheduled_departure, tf.fare_conditions, f.flight_id,
FlightReservationPackage.IsFlightAvailable(  1567, 'Business', '16.07.17
11:30:00,000000000') FROM avia.flights f  JOIN avia.ticket_flights tf ON tf.flight_id =
f.flight_id  WHERE (f.flight_id = 1567
```

```
        AND TO_TIMESTAMP(f.scheduled_departure, 'DD.MM.YY HH24:Mi:SS,FF') =
TO_TIMESTAMP( '16.07.17 11:30:00,000000000', 'DD.MM.YY HH24:Mi:SS,FF')        AND
tf.fare_conditions = 'Business');
--Проверка второй функции
select FlightReservationPackage.GetPassengerReservations('0000 000343') from dual;

--Запросы для отладки
Begin
 FlightReservationPackage.MakeReservation('0000 000343', 336, 'Economy', '12.09.17
10:40:00,000000000');
end;


Begin
 FlightReservationPackage.MakeReservation('8926 645860', 453, 'business', '10.08.17
14:10:00,000000000');
end;
------------------
Begin
 FlightReservationPackage.MakeReservation('0000 000343', 335, 'Business', '16.07.17
11:30:00,000000000');
end;

Begin
 FlightReservationPackage.MakeReservation('4903827' ,1567, 'Economy', '16.07.17
11:30:00,000000000');
end;
-- нет пользователя

Begin
 FlightReservationPackage.MakeReservation('4631 903827' ,1567, 'Economy', '16.07.17
11:30:00,000000000');
end;

--0796 697225 1567 business 03.08.17 10:40:00,000000000
--  1567, 'Business', '16.07.17 11:30:00,000000000'


CREATE OR REPLACE FUNCTION ff(cur_flight_id IN NUMBER, seat_number IN varchar2)
RETURN varchar2
IS
var varchar2(1024);
BEGIN
var:=0;
for elem in
(
select flight_id, seat_no
from AVIA.boarding_passes
```

```
where flight_id = cur_flight_id
) loop
if elem.seat_no = seat_number then
var:= 'занято';
   else
     var:='свободно';
end if;
end loop;
return var;
END ff;

--Проверка доп функции
select flight_id, seat_no, ff(flight_id,seat_no)
  from AVIA.boarding_passes
```

5 Лиза AirportManagementPackage

Создание пакета:

- Создайте пакет «AirportManagementPackage».

Функции:

- Создайте в пакете «AirportManagementPackage» функцию "GenerateTravelReport".

Эта функция должна принимать диапазон дат и возвращать отчет о поездках за этот период. Отчет включает включать общее количество перевезенных пассажиров, список трех популярные направлений (аэропорт прилета) и топ 3 аэропорта с максимальным количество задержанных или отмененных рейсов.

- Создайте в пакете «AirportManagementPackage» функцию "GetAirportHistory".

Эта функция должна принимать ID аэропорта и временной диапазон и возвращать историю вылета рейсов, включая даты и время планового вылета, даты и время фактического вылета рейсов и аэропорты прилета, а также факты отмены рейсов.

Процедура с использованием функций и курсоров:

В пакете «AirportManagementPackage» создайте процедуру "CancelHistory".
Эта процедура должна принимать ID аэропорта вылета и вычислять вероятности отмены или задержки рейса в зависимости от направления вылета. Результаты работы процедуры сохраняются в отдельной таюлице. Процедура должна:

- Проверять наличие указанного аэропорта в нашей базе данных.

• Использовать функцию " GetAirportHistory " за последние 3 года для расчета вероятности задержки рейса или его отмены.

• Если указанный аэропорт не существует или суммарное количество вылетов из него меньше 10-ти, выбросить исключение с четким сообщением об ошибке.

```
CREATE TABLE cancellation_history (
  scheduled_departure TIMESTAMP,
  actual_departure TIMESTAMP,
  arrival_airport VARCHAR2(50) primary key,
  cancellation_status VARCHAR2(10),
  cancellation_probability NUMBER
);
/
CREATE OR REPLACE PACKAGE AirportManagementPackage IS
FUNCTION GenerateTravelReport(start_date IN TIMESTAMP , end_date IN TIMESTAMP )
RETURN VARCHAR2;
 FUNCTION GetAirportHistory(airport_id IN varchar2, start_date IN TIMESTAMP , end_date
IN TIMESTAMP) RETURN SYS_REFCURSOR;
PROCEDURE CancelHistory(departure_airport_id IN varchar2);
END AirportManagementPackage;
/
CREATE OR REPLACE PACKAGE BODY AirportManagementPackage IS
  FUNCTION GenerateTravelReport(start_date IN TIMESTAMP , end_date IN TIMESTAMP )
RETURN VARCHAR2 IS
    v_report VARCHAR2(256);
    rec number;
  BEGIN
    SELECT COUNT(DISTINCT t.passenger_id)
    INTO v_report
    FROM avia.Tickets t
    JOIN avia.Ticket_flights tf ON t.ticket_no = tf.ticket_no
    JOIN avia.Flights f ON tf.flight_id = f.flight_id
    WHERE f.scheduled_departure BETWEEN start_date AND end_date;

    FOR rec IN (select * from (
      SELECT f.arrival_airport, COUNT(DISTINCT t.passenger_id) AS passengers_count
      FROM avia.Tickets t
      JOIN avia.Ticket_flights tf ON t.ticket_no = tf.ticket_no
      JOIN avia.Flights f ON tf.flight_id = f.flight_id
      WHERE f.scheduled_departure BETWEEN start_date AND end_date
      GROUP BY f.arrival_airport
      ORDER BY passengers_count DESC) where rownum <4
    ) LOOP
    EXIT;
      v_report := v_report ||  CHR(10) ||  'Destination: ' ||  rec.arrival_airport  || ', Passengers
Count: ' ||  rec.passengers_count;
    END LOOP;
```

```
  FOR rec IN (
    select * from (

 SELECT departure_airport, COUNT(*) AS flights_count
     FROM AVIA.flights
     WHERE status IN ('Delayed', 'Cancelled')
     GROUP BY departure_airport

     ORDER BY flights_count DESC) where rownum <4
   ) LOOP
   EXIT;
     v_report := v_report ||  CHR(10) ||  'Airport: ' ||  rec.departure_airport ||  ',
Delayed/Cancelled Flights Count: ' ||  rec.flights_count;
   END LOOP;

   RETURN v_report;
  END GenerateTravelReport;


  FUNCTION GetAirportHistory(airport_id IN varchar2, start_date IN TIMESTAMP , end_date
IN TIMESTAMP ) RETURN SYS_REFCURSOR IS
   v_cursor SYS_REFCURSOR;
  BEGIN
   OPEN v_cursor FOR
     SELECT
       f.scheduled_departure AS planned_departure,
       f.actual_departure,
       f.arrival_airport,
       CASE WHEN f.status IN ('Delayed', 'Cancelled') THEN 'YES'
       WHEN f.status Is null THEN 'NO'
       ELSE 'NO' END AS cancellation_status

     FROM avia.Flights f
     WHERE f.departure_airport = airport_id
       AND f.scheduled_departure BETWEEN start_date AND end_date

     ORDER BY f.scheduled_departure;

   RETURN v_cursor;
  END GetAirportHistory;

PROCEDURE CancelHistory(departure_airport_id IN varchar2) IS
  v_total_departures NUMBER;
  v_error_message VARCHAR2(200);
   v_cursor SYS_REFCURSOR :=GetAirportHistory('DME', '16.08.16 09:50:00,000000000',
'16.08.18 09:50:00,000000000');
  v_cancel_probability NUMBER;
```

```
  rec cancellation_history%rowtype;



BEGIN

  SELECT COUNT(*)
  INTO v_total_departures
  FROM avia.Airports_data
  WHERE airport_code = departure_airport_id;
  IF v_total_departures <10 or v_total_departures is null THEN
    v_error_message := 'Аэропорт с ID '||   departure_airport_id  || ' не существует.';
    RAISE_APPLICATION_ERROR(-20001, v_error_message);
  END IF;





    SELECT COUNT(*) INTO v_cancel_probability
    FROM avia.Flights
    WHERE departure_airport = departure_airport_id
      AND status IN ('Delayed', 'Cancelled')
      AND scheduled_departure >= ADD_MONTHS(SYSDATE, -36);

       v_cancel_probability := v_cancel_probability / v_total_departures;
    loop
FETCH v_cursor INTO
    rec.planned_departure,
      rec.actual_departure,
      rec.arrival_airport,
      rec.cancellation_status;
   EXIT WHEN v_cursor%NOTFOUND;

    INSERT INTO cancellation_history (
      scheduled_departure,
      actual_departure,
      arrival_airport,
      cancellation_status,
      cancellation_probability
    ) VALUES (
      rec.planned_departure,
      rec.actual_departure,
      rec.arrival_airport,
      rec.cancellation_status,
      v_cancel_probability
    );
```

```
commit;
END loop;
close v_cursor;
END CancelHistory;
END AirportManagementPackage;
```

6 Даша BookingManagementPackage

Создание пакета:

- Создайте пакет «BookingManagementPackage».

Функции:

- Создайте в пакете «BookingManagementPackage» функцию "GetAvailableFlights". Эта функция должна возвращать список доступных рейсов. Рейсы считаются доступными, если у них есть свободные места и они еще не вылетели или не отменены.

- Создайте в пакете «BookingManagementPackage» функцию "GetFlightDetails". Эта функция должна принимать ID рейса и возвращать детали о рейсе, включая дату и время вылета, прилета, а также аэропорты вылета и прилета. .

Процедура с использованием функций и курсоров:

В пакете " BookingManagementPackage" создайте процедуру "CancelBooking".
Эта процедура должна принимать ID бронирования и отменять соответствующее бронирование. При отмене бронирования место становится снова доступным.
Процедура должна:

- Проверять наличие указанного бронирования.

- Если бронь существует, удалить информацию о ней и входящих в нее билетах из таблицы "Bookings". Удаленные записи необходимо сохранять в промежуточной таблице и после окончания работы вручную добавить обратно. За порчу исходной таблицы штраф (-1 балл за оставление лишних записей после себя, -2 балла за удаления чего-то кроме своих записей из исходных таблиц).

- Использовать явные курсоры для выборки информации о бронировании.

- Проверять, что отмена бронирования происходит более чем за сутки до вылета рейса.

- Использовать функцию " GetFlightDetails " для расчета оставшегося до вылета рейса времени.

• Если указанное бронирование не существует или до вылета рейса осталось меньше суток, выбросить исключение с четким сообщением об ошибке.

```
CREATE OR replace FUNCTION GetFlightDetails(
    p_flight_id IN NUMBER)
  RETURN SYS_REFCURSOR
IS
  v_cursor SYS_REFCURSOR;
BEGIN
  OPEN v_cursor FOR SELECT actual_departure, actual_arrival, departure_airport,
arrival_airport FROM avia.flights WHERE flight_id = p_flight_id AND ROWNUM = 1;
  RETURN v_cursor;
END GetFlightDetails;
```

```
CREATE OR replace FUNCTION GetFlightStatusByDate(
    p_flight_date DATE)
  RETURN SYS_REFCURSOR
IS
  v_cursor SYS_REFCURSOR;
BEGIN
  OPEN v_cursor FOR SELECT f.flight_id, f.departure_airport, f.arrival_airport,
f.scheduled_departure, f.status FROM avia.flights f WHERE TRUNC(f.scheduled_departure)
= p_flight_date;
  RETURN v_cursor;
END GetFlightStatusByDate;
```

```
DECLARE
  v_result SYS_REFCURSOR;
  v_flight_id NUMBER := 335;
  actual_departure DATE;  -- Объявление переменных
  actual_arrival DATE;
  departure_airport VARCHAR2(50);
  arrival_airport VARCHAR2(50);
BEGIN
  v_result := GetFlightDetails(v_flight_id);

  -- Вывод результатов
  DBMS_OUTPUT.PUT_LINE('Flight Details:');

  LOOP
    FETCH v_result INTO actual_departure, actual_arrival, departure_airport, arrival_airport;
    EXIT WHEN v_result%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE('Actual Departure: ' || actual_departure);
```

```
    DBMS_OUTPUT.PUT_LINE('Actual Arrival: ' || actual_arrival);
    DBMS_OUTPUT.PUT_LINE('Departure Airport: ' || departure_airport);
    DBMS_OUTPUT.PUT_LINE('Arrival Airport: ' || arrival_airport);
  END LOOP;

  CLOSE v_result;
END;


DECLARE
  v_result SYS_REFCURSOR;
  v_flight_date DATE := TO_DATE('2017.08.03', 'YYYY.MM.DD'); -- Замените эту дату на
нужную вам
  v_departure_airport VARCHAR2(50) := 'NSK';

  v_flight_id NUMBER;

  v_arrival_airport VARCHAR2(50);
  v_scheduled_departure DATE;
  v_status VARCHAR2(50);
BEGIN
  v_result := GetFlightStatusByDate(v_flight_date);

  -- Вывод результатов
  DBMS_OUTPUT.PUT_LINE('Flight Status on ' || v_flight_date || ':');
  DBMS_OUTPUT.PUT_LINE('Flight ID | Departure Airport | Arrival Airport | Scheduled
Departure | Status');
  DBMS_OUTPUT.PUT_LINE('-----------------------------------------------------------------------------');

  LOOP
    FETCH v_result INTO v_flight_id, v_departure_airport, v_arrival_airport,
v_scheduled_departure, v_status;
    EXIT WHEN v_result%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE(v_flight_id || ' | ' || v_departure_airport || ' | ' || v_arrival_airport
|| ' | ' || v_scheduled_departure || ' | ' || v_status);
  END LOOP;

  CLOSE v_result;

END;




DECLARE
  v_result SYS_REFCURSOR;
  v_flight_date DATE := TO_DATE('2017.08.03', 'YYYY.MM.DD');
```

```
  v_departure_airport VARCHAR2(50) := 'NSK';

  v_flight_id NUMBER;

  v_arrival_airport VARCHAR2(50);
  v_scheduled_departure DATE;
  v_status VARCHAR2(50);
BEGIN
  v_result := GetFlightStatusByDate(v_flight_date, v_departure_city);

  DBMS_OUTPUT.PUT_LINE('Flight Status on ' || v_flight_date || ':');
  DBMS_OUTPUT.PUT_LINE('Flight ID | Departure Airport | Arrival Airport | Scheduled
Departure | Status');
  DBMS_OUTPUT.PUT_LINE('--------------------------------------------------------------------------');

  LOOP
    FETCH v_result INTO v_flight_id, v_departure_airport, v_arrival_airport,
v_scheduled_departure, v_status;
    EXIT WHEN v_result%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE(v_flight_id || ' | ' || v_departure_airport || ' | ' || v_arrival_airport
|| ' | ' || v_scheduled_departure || ' | ' || v_status);
  END LOOP;

  CLOSE v_result;

END;
/




SELECT * FROM AVIA.flights;


CREATE OR REPLACE FUNCTION GetAvailableFlights(p_status VARCHAR2)
RETURN SYS_REFCURSOR
IS
  v_cursor SYS_REFCURSOR;
BEGIN
  OPEN v_cursor FOR
    SELECT f.flight_id, f.status
    FROM avia.flights f
    WHERE f.status = p_status;

  RETURN v_cursor;
END GetAvailableFlights;
```

```
DECLARE
  v_flight_id avia.flights.flight_id%TYPE;
  v_status avia.flights.status%TYPE;
  v_cursor SYS_REFCURSOR;
BEGIN
  v_cursor := GetAvailableFlights('Scheduled'); -- Вызов функции с параметром

  LOOP
    FETCH v_cursor INTO v_flight_id, v_status;
    EXIT WHEN v_cursor%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE('Flight ID: ' || v_flight_id || '  Status: ' || v_status); -- Вывод
идентификатора и статуса рейса
  END LOOP;

  CLOSE v_cursor;
END;
/
```

```
CREATE OR REPLACE PACKAGE BookingManagementPackage AS
   FUNCTION GetFlightDetails(p_flight_id IN NUMBER) RETURN SYS_REFCURSOR;
   FUNCTION GetBookingDetails(p_book_ref IN VARCHAR2) RETURN
SYS_REFCURSOR;
   PROCEDURE CancelBooking(p_book_ref IN VARCHAR2);
END BookingManagementPackage;
```

```
CREATE OR replace FUNCTION GetFlightDetails(
   p_flight_id IN NUMBER)
  RETURN SYS_REFCURSOR
IS
  v_cursor SYS_REFCURSOR;
BEGIN
  OPEN v_cursor FOR SELECT actual_departure, actual_arrival, departure_airport,
arrival_airport FROM avia.flights WHERE flight_id = p_flight_id AND ROWNUM = 1;
  RETURN v_cursor;
END GetFlightDetails;
```

```
CREATE OR replace FUNCTION GetFlightStatusByDate(
   p_flight_date DATE)
  RETURN SYS_REFCURSOR
IS
  v_cursor SYS_REFCURSOR;
BEGIN
  OPEN v_cursor FOR SELECT f.flight_id, f.departure_airport, f.arrival_airport,
f.scheduled_departure, f.status FROM avia.flights f WHERE TRUNC(f.scheduled_departure)
= p_flight_date;
  RETURN v_cursor;
END GetFlightStatusByDate;
```

```
DECLARE
  v_result SYS_REFCURSOR;
  v_flight_id NUMBER := 335;
  actual_departure DATE;  -- Объявление переменных
  actual_arrival DATE;
  departure_airport VARCHAR2(50);
  arrival_airport VARCHAR2(50);
BEGIN
  v_result := GetFlightDetails(v_flight_id);
```

```
  -- Вывод результатов
  DBMS_OUTPUT.PUT_LINE('Flight Details:');

  LOOP
    FETCH v_result INTO actual_departure, actual_arrival, departure_airport, arrival_airport;
    EXIT WHEN v_result%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE('Actual Departure: ' || actual_departure);
    DBMS_OUTPUT.PUT_LINE('Actual Arrival: ' || actual_arrival);
    DBMS_OUTPUT.PUT_LINE('Departure Airport: ' || departure_airport);
    DBMS_OUTPUT.PUT_LINE('Arrival Airport: ' || arrival_airport);
  END LOOP;

  CLOSE v_result;
END;


DECLARE
  v_result SYS_REFCURSOR;
  v_flight_date DATE := TO_DATE('2017.08.03', 'YYYY.MM.DD'); -- Замените эту дату на
нужную вам
  v_departure_airport VARCHAR2(50) := 'NSK';

  v_flight_id NUMBER;

  v_arrival_airport VARCHAR2(50);
  v_scheduled_departure DATE;
  v_status VARCHAR2(50);
BEGIN
  v_result := GetFlightStatusByDate(v_flight_date);

  -- Вывод результатов
  DBMS_OUTPUT.PUT_LINE('Flight Status on ' || v_flight_date || ':');
  DBMS_OUTPUT.PUT_LINE('Flight ID | Departure Airport | Arrival Airport | Scheduled
Departure | Status');
  DBMS_OUTPUT.PUT_LINE('-----------------------------------------------------------------------');

  LOOP
    FETCH v_result INTO v_flight_id, v_departure_airport, v_arrival_airport,
v_scheduled_departure, v_status;
    EXIT WHEN v_result%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE(v_flight_id || ' | ' || v_departure_airport || ' | ' || v_arrival_airport
|| ' | ' || v_scheduled_departure || ' | ' || v_status);
  END LOOP;

  CLOSE v_result;
```

```
END;
```

```
--SELECT * FROM AVIA.flights;
```

```
CREATE OR REPLACE FUNCTION GetAvailableFlights(p_status VARCHAR2)
RETURN SYS_REFCURSOR
IS
  v_cursor SYS_REFCURSOR;
BEGIN
  OPEN v_cursor FOR
    SELECT f.flight_id, f.status
    FROM avia.flights f
    WHERE f.status = p_status;

  RETURN v_cursor;
END GetAvailableFlights;
```

```
DECLARE
  v_flight_id avia.flights.flight_id%TYPE;
  v_status avia.flights.status%TYPE;
  v_cursor SYS_REFCURSOR;
BEGIN
  v_cursor := GetAvailableFlights('Scheduled'); -- Вызов функции с параметром

  LOOP
    FETCH v_cursor INTO v_flight_id, v_status;
    EXIT WHEN v_cursor%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE('Flight ID: ' || v_flight_id || '  Status: ' || v_status); -- Вывод
идентификатора и статуса рейса
  END LOOP;

  CLOSE v_cursor;
END;
/
SET SERVEROUTPUT ON;
```

7 Матвей FlightReservationPackage

```
SELECT * FROM avia.flights;
SELECT * FROM avia.ticket_flights;
SELECT * FROM avia.tickets order by passenger_id;
SELECT * FROM AVIA.bookings;
SELECT * FROM avia.tickets where passenger_id='0000 000343';
SELECT * FROM avia.tickets;
SELECT * FROM avia.seats;
select * from airports;
select * from avia.ticket_flights where flight_id='336';
select * from avia.flights;
select * from avia.boarding_passes;
--1902 281414 336 business 12.09.17 10:40:00,000000000
--453 8926 645860 business 10.08.17 14:10:00,000000000
--id пассажира номер полета класс билета дата
--335 business,economy


CREATE OR REPLACE PACKAGE FlightReservationPackage AS
	FUNCTION IsFlightAvailable(fl_id IN NUMBER, fare_con IN VARCHAR2, flight_date
IN VARCHAR2) RETURN NUMBER;
	FUNCTION GetPassengerReservations(pas_id IN VARCHAR2) RETURN
VARCHAR2;
  PROCEDURE MakeReservation(pas_ids IN VARCHAR2, fl_id IN NUMBER, fare_con IN
VARCHAR2, flight_date IN VARCHAR2);
	FUNCTION check_passenger_ids (p_passenger_ids IN VARCHAR2) RETURN
VARCHAR2;
END FlightReservationPackage;


CREATE OR REPLACE PACKAGE BODY FlightReservationPackage AS
--Создайте в пакете FlightReversationPackage функцию IsFlightAvailable, которая
принимает параметры ID рейса класс билета и дату полета и возвращает количество
доступных билетов.
FUNCTION IsFlightAvailable(fl_id IN NUMBER, fare_con IN VARCHAR2, flight_date IN
VARCHAR2) RETURN NUMBER AS
	available_seats NUMBER;
   BEGIN
      SELECT COUNT(*) INTO available_seats
      FROM avia.flights f
      JOIN avia.ticket_flights tf ON tf.flight_id = f.flight_id
      WHERE (f.flight_id = fl_id
      AND TO_TIMESTAMP(f.scheduled_departure, 'DD.MM.YY HH24:Mi:SS,FF') =
TO_TIMESTAMP(flight_date, 'DD.MM.YY HH24:Mi:SS,FF')
```

```
        AND tf.fare_conditions = fare_con);
                    dbms_output.put_line( fare_con);
        RETURN available_seats;
    END IsFlightAvailable;
--Создайте в пакете FlightReversationPackage функцию GetPassengerReservations,
которая принимает ID пассажира и возвращает информацию о его текущих
бронированиях.
FUNCTION GetPassengerReservations(pas_id IN VARCHAR2) RETURN VARCHAR2 AS
        v_info VARCHAR2 (10000) := '';
        v_book_ref VARCHAR2 (50);
        v_book_date DATE;
        v_total_amount NUMBER;
        v_ticket_no VARCHAR2 (50);
        v_passenger_name VARCHAR2 (50);
        v_contact_data VARCHAR2 (50);
    BEGIN
        FOR rec IN (SELECT b.book_ref, b.book_date, b.total_amount, t.ticket_no,
t.passenger_name, t.contact_data
                FROM AVIA.Bookings b
                JOIN AVIA.Tickets t ON b.book_ref = t.book_ref
                WHERE t.passenger_id = pas_id)
        LOOP
            v_book_ref := rec.book_ref;
            v_book_date := rec.book_date;
            v_total_amount := rec.total_amount;
            v_ticket_no := rec.ticket_no;
            v_passenger_name := rec.passenger_name;
            v_contact_data := rec.contact_data;
            v_info := v_info || 'Booking Reference: ' || v_book_ref || ', Booking Date: ' ||
v_book_date || ', Total Amount: ' || v_total_amount || ', Ticket No: ' || v_ticket_no || ',
Passenger Name: ' || v_passenger_name || ', Contact Data: ' || v_contact_data || CHR(10);
        END LOOP;
        RETURN v_info;
    END GetPassengerReservations;
--В пакете FlightReversationPackage создайте процедуру "MakeReservation", которая
принимает параметры ID пассажира (от 1 до 10), ID рейса, класс билета и дату полета.
--Процедура должна:
--Проверять наличие пассажиров в системе
--Для выборки инофрмации о доступных билетах на указанный рейс и дату и
использовать функцию IsFlightAvailable
--Проверять доступность выбранного класса билетов
--Если билеты доступны добавить новую запись в таблицу бронирования и в билеты
--Если нужное количество билетов не доступно или указанные пассажиры отсутствуют
в базе данных выбросить исключение с четким сообщением об ошибке
PROCEDURE MakeReservation(pas_ids IN VARCHAR2, fl_id IN NUMBER, fare_con IN
VARCHAR2, flight_date IN VARCHAR2) IS

                    v_timer VARCHAR2(30) := TO_DATE(SYSDATE);
```

```
                v_passenger_count NUMBER;
                v_random VARCHAR2(10);
                v_current_pas_id VARCHAR2(20);

        BEGIN

      delete laba_BILETS;
      delete laba_BOOKINGS;
      delete laba_TICKETS;
      commit;


--Ошибка в случае отсутствия пассажира

      if ( (check_passenger_ids(pas_ids) LIKE '%НЕТ%') OR (LENGTH(fl_id) -
LENGTH(REPLACE(fl_id, ',', ''))+1 > IsFlightAvailable(fl_id, fare_con, flight_date)) ) then

        if ( (check_passenger_ids(pas_ids) LIKE '%НЕТ%')) then

                Insert into LOGS( ID_num, PAS_EXIST, COUNT_BILET, PROC_START,
PROC_END , FLAG, oshibka) values( seq_id.nextval, check_passenger_ids(pas_ids),
00000 , v_timer, TO_DATE(SYSDATE), 'НЕТ', 'Такого пользователя не существует!');

        end if;



        if ( (LENGTH(fl_id) - LENGTH(REPLACE(fl_id, ',', ''))+1 > IsFlightAvailable(fl_id,
fare_con, flight_date)) ) then

                Insert into LOGS( ID_num, PAS_EXIST, COUNT_BILET, PROC_START,
PROC_END , FLAG, oshibka) values( seq_id.nextval, check_passenger_ids(pas_ids),
00000 , v_timer, TO_DATE(SYSDATE), 'НЕТ', 'Недостаточно билетов!');

        end if;

      else

                    v_passenger_count := REGEXP_COUNT(pas_ids, ',') + 1;
            dbms_output.put_line(  v_passenger_count);

            FOR oper_rec IN (SELECT tf.TICKET_NO, tf.FARE_CONDITIONS, tf.AMOUNT,
f.FLIGHT_ID, f.FLIGHT_NO, f.SCHEDULED_DEPARTURE, f.SCHEDULED_ARRIVAL,
f.DEPARTURE_AIRPORT, f.ARRIVAL_AIRPORT, f.STATUS, f.AIRCRAFT_CODE
                FROM avia.flights f
                JOIN avia.ticket_flights tf ON tf.flight_id = f.flight_id

                WHERE (f.flight_id = fl_id
```

```
                    AND TO_TIMESTAMP(f.scheduled_departure, 'DD.MM.YY
HH24:Mi:SS,FF') = TO_TIMESTAMP(flight_date, 'DD.MM.YY HH24:Mi:SS,FF')
                    AND tf.fare_conditions = fare_con)
                    ) LOOP



                        insert into laba_bilets values (oper_rec.TICKET_NO,
oper_rec.FARE_CONDITIONS, oper_rec.AMOUNT, oper_rec.FLIGHT_ID,
oper_rec.FLIGHT_NO, oper_rec.SCHEDULED_DEPARTURE,
oper_rec.SCHEDULED_ARRIVAL, oper_rec.DEPARTURE_AIRPORT,
oper_rec.ARRIVAL_AIRPORT, oper_rec.STATUS, oper_rec.AIRCRAFT_CODE);
                        commit;

                        if (v_passenger_count > 0) then

                                        v_current_pas_id:=GetIDFromPosition(pas_ids,
v_passenger_count);
                                        dbms_output.put_line(  v_current_pas_id);
                                        v_random:=SUBSTR(DBMS_RANDOM.STRING('A', 6), 1,
6);
                                        Insert into laba_tickets(ticket_no, book_ref, passenger_id,
passenger_name, contact_data) values( oper_rec.TICKET_NO, v_random,
v_current_pas_id,  getPassengerName(v_current_pas_id),
getContactDataByPassengerId(v_current_pas_id));
                                        Insert into laba_bookings(book_ref, book_date,
TOTAL_AMOUNT) values(v_random, sysdate, oper_rec.AMOUNT);

                                        v_passenger_count := v_passenger_count - 1;

                        end if;


                END LOOP;


                Insert into LOGS( ID_num, PAS_EXIST, COUNT_BILET, PROC_START,
PROC_END , FLAG, OSHIBKA) values( seq_id.nextval, check_passenger_ids(pas_ids),
IsFlightAvailable(fl_id, fare_con, flight_date), v_timer, TO_DATE(SYSDATE), 'ДА', '-');
                Commit;


        end if;

END MakeReservation;
    FUNCTION check_passenger_ids(p_passenger_ids VARCHAR2)
    RETURN VARCHAR2
    IS
```

```
    v_result VARCHAR2(100) := 'ДА';
    v_missing_ids VARCHAR2(100) := '';
    v_count NUMBER;
  BEGIN
    FOR rec IN (
      SELECT REGEXP_SUBSTR(p_passenger_ids, '[^,]+', 1, level) AS passenger_id
      FROM dual
      CONNECT BY REGEXP_SUBSTR(p_passenger_ids, '[^,]+', 1, level) IS NOT NULL
    )
    LOOP
      BEGIN
        SELECT COUNT(*)
        INTO v_count
        FROM avia.Tickets
        WHERE passenger_id = rec.passenger_id;

        IF v_count = 0 THEN
          v_result := 'НЕТ';
          v_missing_ids := v_missing_ids || ',' || rec.passenger_id;
        END IF;
      END;
    END LOOP;

    IF v_result = 'НЕТ' THEN
      v_result := v_result || ' (Пользователь с этим id не найден: ' ||
SUBSTR(v_missing_ids, 2) || ')';
    END IF;

    RETURN v_result;
  END;

END FlightReservationPackage;



--Таблицы для выгрузки операций
create table laba_BILETS as
(SELECT tf.TICKET_NO, tf.FARE_CONDITIONS, tf.AMOUNT, f.FLIGHT_ID, f.FLIGHT_NO,
f.SCHEDULED_DEPARTURE, f.SCHEDULED_ARRIVAL, f.DEPARTURE_AIRPORT,
f.ARRIVAL_AIRPORT, f.STATUS, f.AIRCRAFT_CODE FROM AVIA.flights f
      JOIN AVIA.ticket_flights tf ON tf.flight_id = f.flight_id where rownum<2);
COMMIT;
CREATE TABLE laba_TICKETS AS
(SELECT * FROM AVIA.tickets where rownum<2);
COMMIT;
CREATE TABLE laba_BOOKINGS AS
(SELECT * FROM AVIA.bookings where rownum<2);
COMMIT;
```

```
--Таблица Логов
create table LOGS
  (
    ID_num number primary key,
    PAS_EXIST VARCHAR2(90) not null,
    COUNT_BILET NUMBER not null,

    PROC_START date,
    PROC_END date,

    FLAG VARCHAR2 (30),
    OSHIBKA VARCHAR2 (1000)
  );
  drop table LOGS;


--Последовательность
CREATE SEQUENCE SEQ_ID
START WITH 1
INCREMENT BY 1
NOMAXVALUE;

drop SEQUENCE SEQ_ID;

--Дроп таблиц
DROP TABLE laba_BILETS;
DROP TABLE laba_TICKETS;
DROP TABLE laba_BOOKINGS;
DROP TABLE LOGS;



--Select таблиц
SELECT * FROM laba_BILETS;
SELECT * FROM laba_TICKETS;
SELECT * FROM laba_BOOKINGS;
SELECT * FROM LOGS;




--Запросы для отладки
Begin
 FlightReservationPackage.MakeReservation('4631 903827' ,1567, 'Economy', '16.07.17
11:30:00,000000000');
end;
--0796 697225 1567 business 03.08.17 10:40:00,000000000
--  1567, 'Business', '16.07.17 11:30:00,000000000'
```

```
Begin
 FlightReservationPackage.MakeReservation('0000 000343', 1567, 'Business', '16.07.17
11:30:00,000000000');
end;
-------------------453 8926 645860 business 10.08.17 14:10:00,000000000
Begin
 FlightReservationPackage.MakeReservation('8926 645860', 453, 'business', '10.08.17
14:10:00,000000000');
end;
------------------
Begin
 FlightReservationPackage.MakeReservation('0000 000343', 335, 'Business', '16.07.17
11:30:00,000000000');
end;

Begin
 FlightReservationPackage.MakeReservation('7270 339448,0000 000343' ,1567, 'Economy',
'16.07.17 11:30:00,000000000');
end;


Begin
 FlightReservationPackage.MakeReservation('4903827' ,1567, 'Economy', '16.07.17
11:30:00,000000000');
end;

--1902 281414 336 business 12.09.17 10:40:00,000000000
Begin
 FlightReservationPackage.MakeReservation('1902 281414' ,336, 'business', '12.09.17
10:40:00,000000000');
end;

Begin
 FlightReservationPackage.MakeReservation('4631 903827,7270 339448' ,1567, 'Economy',
'16.07.17 11:30:00,000000000');
end;


Begin
 FlightReservationPackage.MakeReservation('4903827' ,1567, 'Economy', '16.07.17
11:30:00,000000000');
end;


Begin
 FlightReservationPackage.MakeReservation('4631 903827' ,1567, 'Econom', '16.07.17
11:30:00,000000000');
end;
```

```
--Дополнительные функции
CREATE OR REPLACE FUNCTION getPassengerName(passenger_id IN VARCHAR2)
RETURN VARCHAR2 AS
   v_passenger_name VARCHAR2(100);
BEGIN
   SELECT passenger_name INTO v_passenger_name FROM avia.Tickets WHERE
passenger_id = getPassengerName.passenger_id;
         RETURN v_passenger_name;
END;
CREATE OR REPLACE FUNCTION getContactDataByPassengerId(v_passenger_id IN
varchar2) RETURN varchar2 AS
         contact_data_value varchar2(100);
BEGIN
   SELECT contact_data INTO contact_data_value FROM avia.Tickets WHERE
v_passenger_id = passenger_id;
         RETURN contact_data_value;
END;

CREATE OR REPLACE FUNCTION GetIDFromPosition(input_string IN VARCHAR2,
position IN NUMBER) RETURN VARCHAR2 AS
   id_list VARCHAR2(255);
   id VARCHAR2(50);
BEGIN
   id_list := REPLACE(input_string, ',', ' ');
   id := TRIM(REGEXP_SUBSTR(id_list, '\d+ \d+', 1, position));
   RETURN id;
END;



--Проверка доп функций
SELECT GetIDFromPosition('4631 903827,4631 111111', 2) FROM dual;
SELECT SUBSTR(DBMS_RANDOM.STRING('A', 6), 1, 6) FROM dual;
SELECT getPassengerName('4631 903827') FROM dual;
SELECT getContactDataByPassengerId('4631 903827') FROM dual;

--Проверки
--Проверка первой функциии
SELECT DISTINCT f.scheduled_departure, tf.fare_conditions, f.flight_id,
FlightReservationPackage.IsFlightAvailable(  1567, 'Business', '16.07.17
11:30:00,000000000') FROM avia.flights f  JOIN avia.ticket_flights tf ON tf.flight_id =
f.flight_id  WHERE (f.flight_id = 1567
       AND TO_TIMESTAMP(f.scheduled_departure, 'DD.MM.YY HH24:Mi:SS,FF') =
TO_TIMESTAMP( '16.07.17 11:30:00,000000000', 'DD.MM.YY HH24:Mi:SS,FF')       AND
tf.fare_conditions = 'Business');
--Проверка второй функции
SELECT t.passenger_id, FlightReservationPackage.GetPassengerReservations('0000
000343')
```

```
        FROM AVIA.Bookings b
        JOIN AVIA.Tickets t ON b.book_ref = t.book_ref
        WHERE t.passenger_id = '0000 000343';
```

```
select FlightReservationPackage.check_passenger_ids('0796 697225,123,12111') from dual;
```

```
CREATE OR REPLACE FUNCTION GetSeatClass(v_flight_number IN VARCHAR2,
v_seat_number IN VARCHAR2) RETURN VARCHAR2 AS
   seat_class VARCHAR2(20);
BEGIN
   SELECT CASE
         WHEN s.fare_conditions = 'Business' THEN 'Бизнес'
         WHEN s.fare_conditions = 'Economy' THEN 'Эконом'
         ELSE 'Неизвестно'
      END
   INTO seat_class
   FROM avia.seats s
   WHERE s.aircraft_code = v_flight_number
   AND s.seat_no = v_seat_number;

   RETURN seat_class;
EXCEPTION
   WHEN NO_DATA_FOUND THEN
      RETURN 'Неизвестно';
END;
```

```
SELECT GetSeatClass('773', '38B') AS seat_class FROM dual;
```

```
SET SERVEROUTPUT ON;
```

8 Шестик FlightReservationPackage

Создание пакета:

- Создайте пакет "FlightReservationPackage".

Функции:

- Создайте в пакете "FlightReservationPackage" функцию "IsFlightAvailable", которая принимает параметры ID рейса, класс билета и дату полета, и возвращает количество доступных билетов. Функция должна проверять доступность билетов выбранного класса на указанный рейс и дату.

- Создайте в пакете "FlightReservationPackage" функцию "GetPassengerReservations", которая принимает ID пассажира и возвращает информацию о его текущих бронированиях.

Процедура с использованием функций и курсоров:

- В пакете "FlightReservationPackage" создайте процедуру "MakeReservation", которая принимает параметры: ID пассажиров (от 1 до 10), ID рейса, класс билета и дату полета. Процедура должна:

- Проверять наличие пассажиров в системе.

- Для выборки информации о доступных билетах на указанный рейс и дату с использовать функцию "IsFlightAvailable".

- Проверять доступность выбранного класса билетов.

- Если билеты доступны, добавить новую запись в таблицу бронирования и в билеты. После завершения работы все добавленные записи необходимо удалить. За порчу исходной таблицы штраф (-1 балл за оставление лишних записей после себя, -2 балла за удаления чего-то кроме своих записей из исходных таблиц).

- Если нужное количество билетов не доступно или указанные пассажиры отсуствуют в базе данных выбросить исключение с четким сообщением об ошибке.

```
SELECT * FROM avia.flights;
SELECT * FROM avia.ticket_flights;
SELECT * FROM avia.tickets order by passenger_id;
SELECT * FROM AVIA.bookings;
SELECT * FROM avia.tickets where passenger_id='0000 000343';
SELECT * FROM avia.tickets;
SELECT * FROM avia.seats;
select * from airports;
select * from avia.ticket_flights where flight_id='336';
select * from avia.flights;
select * from avia.boarding_passes;
--1902 281414 336 business 12.09.17 10:40:00,000000000
--453 8926 645860 business 10.08.17 14:10:00,000000000
--id пассажира номер полета класс билета дата
--335 business,economy
```

```
CREATE OR REPLACE PACKAGE FlightReservationPackage AS
	FUNCTION IsFlightAvailable(fl_id IN NUMBER, fare_con IN VARCHAR2, flight_date
IN VARCHAR2) RETURN NUMBER;
	FUNCTION GetPassengerReservations(pas_id IN VARCHAR2) RETURN
VARCHAR2;
  PROCEDURE MakeReservation(pas_ids IN VARCHAR2, fl_id IN NUMBER, fare_con IN
VARCHAR2, flight_date IN VARCHAR2);
	FUNCTION get_seat_no_by_ticket_no(p_ticket_no IN NUMBER) RETURN
VARCHAR2;
END FlightReservationPackage;


CREATE OR REPLACE PACKAGE BODY FlightReservationPackage AS
--Создайте в пакете FlightReversationPackage функцию IsFlightAvailable, которая
принимает параметры ID рейса класс билета и дату полета и возвращает количество
доступных билетов.
FUNCTION IsFlightAvailable(fl_id IN NUMBER, fare_con IN VARCHAR2, flight_date IN
VARCHAR2) RETURN NUMBER AS
	available_seats NUMBER;
   BEGIN
      SELECT COUNT(*) INTO available_seats
      FROM avia.flights f
      JOIN avia.ticket_flights tf ON tf.flight_id = f.flight_id
      WHERE (f.flight_id = fl_id
      AND TO_TIMESTAMP(f.scheduled_departure, 'DD.MM.YY HH24:Mi:SS,FF') =
TO_TIMESTAMP(flight_date, 'DD.MM.YY HH24:Mi:SS,FF')
      AND tf.fare_conditions = fare_con);
                dbms_output.put_line( fare_con);
      RETURN available_seats;
   END IsFlightAvailable;
--Создайте в пакете FlightReversationPackage функцию GetPassengerReservations,
которая принимает ID пассажира и возвращает информацию о его текущих
бронированиях.
FUNCTION GetPassengerReservations(pas_id IN VARCHAR2) RETURN VARCHAR2 AS
      v_info VARCHAR2 (10000) := '';
      v_book_ref VARCHAR2 (50);
      v_book_date DATE;
      v_total_amount NUMBER;
      v_ticket_no VARCHAR2 (50);
      v_passenger_name VARCHAR2 (50);
      v_contact_data VARCHAR2 (50);
   BEGIN
      FOR rec IN (SELECT b.book_ref, b.book_date, b.total_amount, t.ticket_no,
t.passenger_name, t.contact_data
                FROM AVIA.Bookings b
                JOIN AVIA.Tickets t ON b.book_ref = t.book_ref
                WHERE t.passenger_id = pas_id)
```

```
        LOOP
            v_book_ref := rec.book_ref;
            v_book_date := rec.book_date;
            v_total_amount := rec.total_amount;
            v_ticket_no := rec.ticket_no;
            v_passenger_name := rec.passenger_name;
            v_contact_data := rec.contact_data;
            v_info := v_info || 'Booking Reference: ' || v_book_ref || ', Booking Date: ' ||
v_book_date || ', Total Amount: ' || v_total_amount || ', Ticket No: ' || v_ticket_no || ',
Passenger Name: ' || v_passenger_name || ', Contact Data: ' || v_contact_data || CHR(10);
        END LOOP;
        RETURN v_info;
    END GetPassengerReservations;
--В пакете FlightReversationPackage создайте процедуру "MakeReservation", которая
принимает параметры ID пассажира (от 1 до 10), ID рейса, класс билета и дату полета.
--Процедура должна:
--Проверять наличие пассажиров в системе
--Для выборки инофрмации о доступных билетах на указанный рейс и дату и
использовать функцию IsFlightAvailable
--Проверять доступность выбранного класса билетов
--Если билеты доступны добавить новую запись в таблицу бронирования и в билеты
--Если нужное количество билетов не доступно или указанные пассажиры отсутствуют
в базе данных выбросить исключение с четким сообщением об ошибке
PROCEDURE MakeReservation(pas_ids IN VARCHAR2, fl_id IN NUMBER, fare_con IN
VARCHAR2, flight_date IN VARCHAR2) IS

                v_timer VARCHAR2(30) := TO_DATE(SYSDATE);
                v_passenger_count NUMBER;
                v_random VARCHAR2(10);
                v_current_pas_id VARCHAR2(20);

            BEGIN

        delete SHESTOPEROV_BILETS;
        delete SHESTOPEROV_BOOKINGS;
        delete SHESTOPEROV_TICKETS;
        commit;


--Ошибка в случае отсутствия пассажира

        if ( (check_passenger_ids(pas_ids) LIKE '%HET%') OR (LENGTH(fl_id) -
LENGTH(REPLACE(fl_id, ',', ''))+1 > IsFlightAvailable(fl_id, fare_con, flight_date)) ) then

            if ( (check_passenger_ids(pas_ids) LIKE '%HET%')) then
```

```
                Insert into LOGS( ID_num, PAS_EXIST, COUNT_BILET, PROC_START,
PROC_END , FLAG, oshibka) values( seq_id.nextval, check_passenger_ids(pas_ids),
00000 , v_timer, TO_DATE(SYSDATE), 'НЕТ', 'Такого пользователя не существует!');

        end if;



        if ( (LENGTH(fl_id) - LENGTH(REPLACE(fl_id, ',', ''))+1 > IsFlightAvailable(fl_id,
fare_con, flight_date)) ) then

                Insert into LOGS( ID_num, PAS_EXIST, COUNT_BILET, PROC_START,
PROC_END , FLAG, oshibka) values( seq_id.nextval, check_passenger_ids(pas_ids),
00000 , v_timer, TO_DATE(SYSDATE), 'НЕТ', 'Недостаточно билетов!');

        end if;


      else

                        v_passenger_count := REGEXP_COUNT(pas_ids, ',') + 1;
            dbms_output.put_line(  v_passenger_count);

            FOR oper_rec IN (SELECT tf.TICKET_NO, tf.FARE_CONDITIONS, tf.AMOUNT,
f.FLIGHT_ID, f.FLIGHT_NO, f.SCHEDULED_DEPARTURE, f.SCHEDULED_ARRIVAL,
f.DEPARTURE_AIRPORT, f.ARRIVAL_AIRPORT, f.STATUS, f.AIRCRAFT_CODE
                    FROM avia.flights f
                    JOIN avia.ticket_flights tf ON tf.flight_id = f.flight_id

                    WHERE (f.flight_id = fl_id
                    AND TO_TIMESTAMP(f.scheduled_departure, 'DD.MM.YY
HH24:Mi:SS,FF') = TO_TIMESTAMP(flight_date, 'DD.MM.YY HH24:Mi:SS,FF')
                    AND tf.fare_conditions = fare_con)
                    ) LOOP



                        insert into ogorodnikov_bilets values (oper_rec.TICKET_NO,
oper_rec.FARE_CONDITIONS, oper_rec.AMOUNT, oper_rec.FLIGHT_ID,
oper_rec.FLIGHT_NO, oper_rec.SCHEDULED_DEPARTURE,
oper_rec.SCHEDULED_ARRIVAL, oper_rec.DEPARTURE_AIRPORT,
oper_rec.ARRIVAL_AIRPORT, oper_rec.STATUS, oper_rec.AIRCRAFT_CODE);
                        commit;

                        if (v_passenger_count > 0) then

                                        v_current_pas_id:=GetIDFromPosition(pas_ids,
v_passenger_count);
```

```
                                        dbms_output.put_line(  v_current_pas_id);
                                        v_random:=SUBSTR(DBMS_RANDOM.STRING('A', 6), 1,
6);
                                        Insert into ogorodnikov_tickets(ticket_no, book_ref,
passenger_id, passenger_name, contact_data) values( oper_rec.TICKET_NO, v_random,
v_current_pas_id,  getPassengerName(v_current_pas_id),
getContactDataByPassengerId(v_current_pas_id));
                                        Insert into ogorodnikov_bookings(book_ref, book_date,
TOTAL_AMOUNT) values(v_random, sysdate, oper_rec.AMOUNT);

                                        v_passenger_count := v_passenger_count - 1;

                        end if;


                END LOOP;


                    Insert into LOGS( ID_num, PAS_EXIST, COUNT_BILET, PROC_START,
PROC_END , FLAG, OSHIBKA) values( seq_id.nextval, check_passenger_ids(pas_ids),
IsFlightAvailable(fl_id, fare_con, flight_date), v_timer, TO_DATE(SYSDATE), 'ДА', '-');
                    Commit;


        end if;

END MakeReservation;
  FUNCTION get_seat_no_by_ticket_no(p_ticket_no IN NUMBER)
RETURN VARCHAR2
IS
   v_seat_no VARCHAR2(20);
BEGIN
   SELECT bp.seat_no
   INTO v_seat_no
   FROM Boarding_passes bp
   JOIN Ticket_flights tf ON bp.ticket_no = tf.ticket_no
   WHERE tf.ticket_no = p_ticket_no;
         AND bp.flight_id = tf.flight_id

   RETURN v_seat_no;
EXCEPTION
   WHEN NO_DATA_FOUND THEN
      RETURN 'Seat not found';
END;

END FlightReservationPackage;
```

```
--Таблицы для выгрузки операций
create table SHESTOPEROV_BILETS as
(SELECT tf.TICKET_NO, tf.FARE_CONDITIONS, tf.AMOUNT, f.FLIGHT_ID, f.FLIGHT_NO,
f.SCHEDULED_DEPARTURE, f.SCHEDULED_ARRIVAL, f.DEPARTURE_AIRPORT,
f.ARRIVAL_AIRPORT, f.STATUS, f.AIRCRAFT_CODE FROM AVIA.flights f
        JOIN AVIA.ticket_flights tf ON tf.flight_id = f.flight_id where rownum<2);
COMMIT;
CREATE TABLE SHESTOPEROV_TICKETS AS
(SELECT * FROM AVIA.tickets where rownum<2);
COMMIT;
CREATE TABLE SHESTOPEROV_BOOKINGS AS
(SELECT * FROM AVIA.bookings where rownum<2);
COMMIT;
--Таблица Логов
create table LOGS
  (
    ID_num number primary key,
    PAS_EXIST VARCHAR2(50) not null,
    COUNT_BILET NUMBER not null,

    PROC_START date,
    PROC_END date,

    FLAG VARCHAR2 (30),
    OSHIBKA VARCHAR2 (1000)
  );
  drop table LOGS;


--Последовательность
CREATE SEQUENCE SEQ_ID
START WITH 1
INCREMENT BY 1
NOMAXVALUE;



--Дроп таблиц
DROP TABLE SHESTOPEROV_BILETS;
DROP TABLE SHESTOPEROV_TICKETS;
DROP TABLE SHESTOPEROV_BOOKINGS;
DROP TABLE LOGS;



--Select таблиц
SELECT * FROM SHESTOPEROV_BILETS;
```

```
SELECT * FROM SHESTOPEROV_TICKETS;
SELECT * FROM SHESTOPEROV_BOOKINGS;
SELECT * FROM LOGS;



--Запросы для отладки
Begin
 FlightReservationPackage.MakeReservation('4631 903827' ,1567, 'Economy', '16.07.17
11:30:00,000000000');
end;
--0796 697225 1567 business 03.08.17 10:40:00,000000000
--  1567, 'Business', '16.07.17 11:30:00,000000000'
Begin
 FlightReservationPackage.MakeReservation('0000 000343', 1567, 'Business', '16.07.17
11:30:00,000000000');
end;
-------------------453 8926 645860 business 10.08.17 14:10:00,000000000
Begin
 FlightReservationPackage.MakeReservation('8926 645860', 453, 'business', '10.08.17
14:10:00,000000000');
end;
------------------
Begin
 FlightReservationPackage.MakeReservation('0000 000343', 335, 'Business', '16.07.17
11:30:00,000000000');
end;

Begin
 FlightReservationPackage.MakeReservation('4631 903827,7270 339448' ,1567, 'Economy',
'16.07.17 11:30:00,000000000');
end;


Begin
 FlightReservationPackage.MakeReservation('4903827' ,1567, 'Economy', '16.07.17
11:30:00,000000000');
end;

--1902 281414 336 business 12.09.17 10:40:00,000000000
Begin
 FlightReservationPackage.MakeReservation('1902 281414' ,336, 'business', '12.09.17
10:40:00,000000000');
end;

Begin
```

```
 FlightReservationPackage.MakeReservation('4631 903827,7270 339448' ,1567, 'Economy',
'16.07.17 11:30:00,000000000');
end;


Begin
 FlightReservationPackage.MakeReservation('4903827' ,1567, 'Economy', '16.07.17
11:30:00,000000000');
end;


Begin
 FlightReservationPackage.MakeReservation('4631 903827' ,1567, 'Econom', '16.07.17
11:30:00,000000000');
end;
--Дополнительные функции
CREATE OR REPLACE FUNCTION getPassengerName(passenger_id IN VARCHAR2)
RETURN VARCHAR2 AS
   v_passenger_name VARCHAR2(100);
BEGIN
   SELECT passenger_name INTO v_passenger_name FROM avia.Tickets WHERE
passenger_id = getPassengerName.passenger_id;
         RETURN v_passenger_name;
END;
CREATE OR REPLACE FUNCTION getContactDataByPassengerId(v_passenger_id IN
varchar2) RETURN varchar2 AS
         contact_data_value varchar2(100);
BEGIN
   SELECT contact_data INTO contact_data_value FROM avia.Tickets WHERE
v_passenger_id = passenger_id;
         RETURN contact_data_value;
END;
CREATE OR REPLACE FUNCTION GetIDFromPosition(input_string IN VARCHAR2,
position IN NUMBER) RETURN VARCHAR2 AS
   id_list VARCHAR2(255);
   id VARCHAR2(50);
BEGIN
   id_list := REPLACE(input_string, ',', ' ');
   id := TRIM(REGEXP_SUBSTR(id_list, '\d+ \d+', 1, position));
   RETURN id;
END;



--Проверка доп функций
SELECT GetIDFromPosition('4631 903827,4631 111111', 2) FROM dual;
SELECT SUBSTR(DBMS_RANDOM.STRING('A', 6), 1, 6) FROM dual;
SELECT getPassengerName('4631 903827') FROM dual;
```

```
SELECT getContactDataByPassengerId('4631 903827') FROM dual;

--Проверки
--Проверка первой функциии
SELECT f.scheduled_departure, tf.fare_conditions, f.flight_id,
FlightReservationPackage.IsFlightAvailable(  1567, 'Business', '16.07.17
11:30:00,000000000') FROM avia.flights f  JOIN avia.ticket_flights tf ON tf.flight_id =
f.flight_id  WHERE (f.flight_id = 1567
       AND TO_TIMESTAMP(f.scheduled_departure, 'DD.MM.YY HH24:Mi:SS,FF') =
TO_TIMESTAMP( '16.07.17 11:30:00,000000000', 'DD.MM.YY HH24:Mi:SS,FF')       AND
tf.fare_conditions = 'Business');
--Проверка второй функции
SELECT t.passenger_id, FlightReservationPackage.GetPassengerReservations('4631
903827')
              FROM AVIA.Bookings b
              JOIN AVIA.Tickets t ON b.book_ref = t.book_ref
              WHERE t.passenger_id = '4631 903827';


--Проверка третьей функции

select FlightReservationPackage.check_passenger_ids('0796 697225,123,12111') from dual;

select  passenger_id, FlightReservationPackage.check_passenger_ids(passenger_id)  from
avia.tickets;


SET SERVEROUTPUT ON;
```

9 Леша BookingManagementPackage

Создание пакета:

- Создайте пакет «BookingManagementPackage».

Функции:

- Создайте в пакете «BookingManagementPackage» функцию "GetAvailableFlights". Эта функция должна возвращать список доступных рейсов. Рейсы считаются доступными, если у них есть свободные места и они еще не вылетели или не отменены.

- Создайте в пакете «BookingManagementPackage» функцию "GetFlightDetails". Эта функция должна принимать ID рейса и возвращать детали о рейсе, включая дату и время вылета, прилета, а также аэропорты вылета и прилета. .

Процедура с использованием функций и курсоров:

В пакете " BookingManagementPackage" создайте процедуру "CancelBooking".
Эта процедура должна принимать ID бронирования и отменять соответствующее бронирование. При отмене бронирования место становится снова доступным.
Процедура должна:

- Проверять наличие указанного бронирования.

- Если бронь существует, удалить информацию о ней и входящих в нее билетах из таблицы "Bookings". Удаленные записи необходимо сохранять в промежуточной таблице и после окончания работы вручную добавить обратно. За порчу исходной таблицы штраф (-1 балл за оставление лишних записей после себя, -2 балла за удаления чего-то кроме своих записей из исходных таблиц).

- Использовать явные курсоры для выборки информации о бронировании.

- Проверять, что отмена бронирования происходит более чем за сутки до вылета рейса.

- Использовать функцию " GetFlightDetails " для расчета оставшегося до вылета рейса времени.

- Если указанное бронирование не существует или до вылета рейса осталось меньше суток, выбросить исключение с четким сообщением об ошибке.

```
SELECT table_name FROM all_tables WHERE owner='AVIA';

select *
from AVIA.bookings;

select *
from AVIA.aircrafts_data;

select *
from AVIA.airports_data;

select *
from AVIA.boarding_passes;

select *
from AVIA.flights;

select *
from AVIA.seats;
```

```
select *
from AVIA.tickets;

select *
from AVIA.ticket_flights;

create or replace package BookingManagementPackage as
FUNCTION GetAvailableFlights return sys_refcursor;
FUNCTION GetFlightDetails (flightId INTEGER) return sys_refcursor;
end BookingManagementPackage;

create or replace package body BookingManagementPackage as
function GetAvailableFlights return sys_refcursor is
rc sys_refcursor;
begin
  open rc for
    select distinct fl.flight_id, s.seat_no, fl.status
    from avia.flights fl
    join avia.aircrafts_data ad on ad.aircraft_code = fl.aircraft_code
    join avia.seats s on s.aircraft_code = ad.aircraft_code
    left join avia.boarding_passes bp on bp.seat_no = s.seat_no
    where bp.seat_no is null and  fl.flight_id = bp.flight_id and fl.status not in
    ('Cancelled', 'Departed', 'Arrived');
  return rc;
end GetAvailableFlights;
function GetFlightDetails (flightId INTEGER) return sys_refcursor is
rc1 sys_refcursor;
begin
  open rc1 for
    select fl.flight_id, fl.scheduled_departure, fl.scheduled_arrival,
      ap.airport_name as departure_airport_name, ap1.airport_name as arrival_airport_name
    from avia.flights fl
    join avia.airports_data ap on ap.airport_code = fl.departure_airport
    join avia.airports_data ap1 on ap1.airport_code = fl.arrival_airport
    where fl.flight_id = :flightId;
  return rc1;
end GetFlightDetails;
end BookingManagementPackage;

create or replace function GetAvailableFlights return sys_refcursor is
rc sys_refcursor;
begin
  open rc for
    select distinct fl.flight_id, s.seat_no, fl.status
    from avia.flights fl
    join avia.aircrafts_data ad on ad.aircraft_code = fl.aircraft_code
    join avia.seats s on s.aircraft_code = ad.aircraft_code
```

```
    left join avia.boarding_passes bp on bp.seat_no = s.seat_no
    where bp.seat_no is null and  fl.flight_id = bp.flight_id and fl.status not in
    ('Cancelled', 'Departed', 'Arrived');
  return rc;
end;

-- есть свободные места и они еще не вылетели или не отменены
select distinct fl.flight_id, s.seat_no, fl.status
from avia.flights fl
join avia.aircrafts_data ad on ad.aircraft_code = fl.aircraft_code
join avia.seats s on s.aircraft_code = ad.aircraft_code
left join avia.boarding_passes bp on bp.seat_no = s.seat_no
where bp.seat_no is null and  fl.flight_id = bp.flight_id and fl.status not in
('Cancelled', 'Departed', 'Arrived');

--GetFlightDetails возвращать детали о рейсе,
--включая дату и время вылета, прилета, а также аэропорты вылета и прилета
select fl.flight_id, fl.scheduled_departure, fl.scheduled_arrival,
  ap.airport_name as departure_airport_name, ap1.airport_name as arrival_airport_name
from avia.flights fl
join avia.airports_data ap on ap.airport_code = fl.departure_airport
join avia.airports_data ap1 on ap1.airport_code = fl.arrival_airport
where fl.flight_id = :flightId;

select *
from AVIA.boarding_passes bp
left join avia.tickets t on bp.ticket_no = t.ticket_no
where t.ticket_no is null;

create or replace function GetFlightDetails (flightId in number) return sys_refcursor is
rc1 sys_refcursor;
begin
  open rc1 for
    select fl.flight_id, fl.scheduled_departure, fl.scheduled_arrival,
      ap.airport_name as departure_airport_name, ap1.airport_name as arrival_airport_name
    from avia.flights fl
    join avia.airports_data ap on ap.airport_code = fl.departure_airport
    join avia.airports_data ap1 on ap1.airport_code = fl.arrival_airport
    where fl.flight_id = flightId;
  return rc1;
end GetFlightDetails;

select GetFlightDetails(fl.flight_id)
from avia.flights fl
where fl.flight_id = 321;
--новое
--
select *
```

```
from avia.bookings
where book_ref = '00E411';

--CancelBooking
create or replace procedure CancelBooking (book_ref in varchar2)
is
cnumber number;
cursor cur is
select b.book_ref, b.book_date, b.total_amount, t.passenger_id,
t.passenger_name, t.contact_data
from avia.bookings b
join avia.tickets t on t.book_ref = b.book_ref
where b.book_ref = book_ref;
begin
  savepoint my_start;
  for booking in cur
  LOOP
    insert into deleted_bookings(id, book_ref, book_date, total_amount)
    values(LogAutoIncrement.nextval, booking.book_ref,
    booking.book_date, booking.total_amount);

    delete from avia.bookings b where b.book_ref = book_ref;
    --нужно доп.цикл сделать для каждого - нужен доп.курсор
    insert into deleted_tickets(ticket_no, book_ref,
    passenger_id, passenger_name, contact_data)
    values(booking.ticket_no, booking.book_ref,
    booking.passenger_id, booking.passenger_name,
    booking.contact_data);

    delete from avia.tickets where
  END LOOP;
  commit;
EXCEPTION
when others then
  rollback to my_start;
end;

create table deleted_bookings (
  id number,
  book_ref varchar2(255),
  book_date date,
  total_amount number
);

create table deleted_tickets (
  ticket_no varchar2(255),
  book_ref varchar2(255),
  passenger_id varchar2(255),
```

```
  passenger_name varchar2(255),
  contact_data varchar2(255)
);

create or replace procedure CancelBooking (book_ref_id in varchar2)
is
cnumber number;
book_ref_name varchar2(255);
cursor cur is select b.book_ref, b.book_date, b.total_amount, t.passenger_id,
t.passenger_name, t.contact_data, t.ticket_no, tf.flight_id, tf.fare_conditions,
tf.amount as tick_flight_amount, bp.boarding_no, bp.seat_no
from avia.bookings b
join avia.tickets t on t.book_ref = b.book_ref
join avia.ticket_flights tf on tf.ticket_no = t.ticket_no
join avia.boarding_passes bp on bp.ticket_no = t.ticket_no
where b.book_ref = book_ref_id;
begin
  --savepoint my_start;
  for booking in cur
    LOOP
     insert into deleted_boarding_passes(ticket_no,
      flight_id, boarding_no, seat_no)
      values (booking.ticket_no, booking.flight_id, booking.boarding_no,
      booking.seat_no);
      delete avia.boarding_passes where ticket_no = booking.ticket_no;

      insert into deleted_ticket_flights(ticket_no, flight_id,
      fare_conditions, amount)
      values(booking.ticket_no, booking.flight_id, booking.fare_conditions,
      booking.tick_flight_amount);
      delete avia.ticket_flights where ticket_no = booking.ticket_no;

      insert into deleted_tickets(ticket_no, book_ref,
      passenger_id, passenger_name, contact_data)
      values(booking.ticket_no, booking.book_ref,
      booking.passenger_id, booking.passenger_name,
      booking.contact_data);
      delete avia.tickets where ticket_no = booking.ticket_no;

    --нужно доп.цикл сделать для каждого - нужен доп.курсор
    END LOOP;
    insert into deleted_bookings select book_ref, book_date, total_amount
    from avia.bookings where book_ref = book_ref_id;

    delete from avia.bookings b where b.book_ref = book_ref_id;
  --commit;
--EXCEPTION
--when others then
```

```
  --rollback to my_start;
end;

create table deleted_bookings (
  book_ref varchar2(255),
  book_date date,
  total_amount number
);

create table deleted_tickets (
  ticket_no varchar2(255),
  book_ref varchar2(255),
  passenger_id varchar2(255),
  passenger_name varchar2(255),
  contact_data varchar2(255)
);

create table deleted_ticket_flights (
  ticket_no varchar2(255),
  flight_id varchar2(255),
  fare_conditions varchar2(255),
  amount number
);

create table deleted_boarding_passes (
  ticket_no varchar2(255),
  flight_id varchar2(255),
  boarding_no varchar2(255),
  seat_no varchar2(255)
);
```

10 София AirportManagementPackage

```
CREATE OR REPLACE PACKAGE AirportManagementPackage AUTHID DEFINER AS
    TYPE report_cursor IS REF CURSOR; -- курсор для возврата отчетных данных
    TYPE airport_cursor IS REF CURSOR; -- курсор для работы с историей рейсов по
аэропорту
-- возвращает отчёт о поездках за период
FUNCTION GenerateTravelReport(
    start_date DATE,
    end_date DATE
) RETURN SYS_REFCURSOR;
-- возвращает историю рейсов из аэропорта
FUNCTION GetAirportHistory(
    p_airport_id VARCHAR2,
    p_start_date DATE,
    p_end_date DATE
```

```
) RETURN SYS_REFCURSOR;
-- вычисление вероятности отмены или задержки рейса в зависимости от направления
вылета
PROCEDURE CancelHistory(
   p_airport_id VARCHAR2
);
-- Моя функция: рассчет сред время полета и среднюю задержку
FUNCTION CalculateFlightTime(
   p_departure_airport VARCHAR2, -- вылет
   p_arrival_airport VARCHAR2 -- прилет
) RETURN VARCHAR2; -- возращает строку с отчетом
END AirportManagementPackage;


CREATE TABLE cancel_history_stats (
  run_id            NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  airport_id        VARCHAR2(20)    NOT NULL,
  analysis_date     DATE            DEFAULT SYSDATE NOT NULL,
  total_flights     NUMBER          NOT NULL,
  cancelled_flights  NUMBER         NOT NULL,
  delayed_flights    NUMBER         NOT NULL,
  cancelled_pct     NUMBER(5,2)     NOT NULL,
  delayed_pct       NUMBER(5,2)     NOT NULL
);

----
CREATE OR REPLACE PACKAGE BODY AirportManagementPackage AS
   FUNCTION GenerateTravelReport(
      start_date DATE,
      end_date DATE
   ) RETURN SYS_REFCURSOR
   IS
      report_cursor SYS_REFCURSOR;
   BEGIN
      OPEN report_cursor FOR
         WITH FlightStats AS ( -- общее кол-во пассажиров за период
            SELECT COUNT(DISTINCT t.passenger_id) AS total_passengers
            FROM avia.flights f
            JOIN avia.ticket_flights tf ON f.flight_id = tf.flight_id
            JOIN avia.tickets t ON tf.ticket_no = t.ticket_no
            WHERE f.scheduled_departure BETWEEN start_date AND end_date
         ),
         PopularDestinations AS ( -- топ 3 направлений (аэропорт прилета) по кол-ву
рейсов
            SELECT f.arrival_airport, COUNT(*) AS flight_count
            FROM avia.flights f
            WHERE f.scheduled_departure BETWEEN start_date AND end_date
            GROUP BY f.arrival_airport
```

```
            ORDER BY flight_count DESC
            FETCH FIRST 3 ROWS ONLY
        ),
        DelayedOrCancelled AS ( -- топ 3 аэропорта вылета с задержками/отменами
            SELECT f.departure_airport, COUNT(*) AS delay_cancel_count
            FROM avia.flights f
            WHERE f.scheduled_departure BETWEEN start_date AND end_date
            AND (f.status IN ('Delayed', 'Cancelled'))
            GROUP BY f.departure_airport
            ORDER BY delay_cancel_count DESC
            FETCH FIRST 3 ROWS ONLY
        )
        ---- в отчет объединяю пассажиры/топ-напрвления/задержки
        SELECT
            'Total Passengers' AS report_type,
            CAST(fs.total_passengers AS VARCHAR2(50)) AS report_value -- перевожу
число в текст
        FROM FlightStats fs
        UNION ALL
        SELECT
            'Popular Destination: ' || pd.arrival_airport,
            CAST(pd.flight_count AS VARCHAR2(50))
        FROM PopularDestinations pd
        UNION ALL
        SELECT
            'Most Delayed/Cancelled: ' || dc.departure_airport,
            CAST(dc.delay_cancel_count AS VARCHAR2(50))
        FROM DelayedOrCancelled dc;
    RETURN report_cursor;
  END GenerateTravelReport;
--  расчет задержки
  FUNCTION GetAirportHistory(
    p_airport_id VARCHAR2,
    p_start_date DATE,
    p_end_date DATE
  ) RETURN SYS_REFCURSOR
  IS
    airport_cursor SYS_REFCURSOR;
  BEGIN
    OPEN airport_cursor FOR
        SELECT
            flight_no,
            scheduled_departure, --плановое время
            actual_departure, --фактич время
            arrival_airport,--код аэропорта
            status
        FROM avia.flights
        WHERE departure_airport = p_airport_id
```

```
        AND scheduled_departure BETWEEN p_start_date AND p_end_date
        ORDER BY scheduled_departure;
      RETURN airport_cursor;
   END GetAirportHistory;
--
PROCEDURE CancelHistory(
   p_airport_id VARCHAR2
)
IS
   v_total_flights NUMBER := 0; -- всего рейсов
   v_cancelled_flights NUMBER := 0; -- колич отмененных
   v_delayed_flights NUMBER := 0; -- колич задержанных
   airport_cursor SYS_REFCURSOR; -- курсор с историей рейсов
   v_flight_no VARCHAR2(20); --
   v_scheduled_departure DATE;
   v_actual_departure DATE;
   v_arrival_airport VARCHAR2(10);
   v_status VARCHAR2(20);
   v_airport_exists NUMBER; -- существует ли аэропорт

BEGIN
   SELECT COUNT(*) INTO v_airport_exists
   FROM avia.airports_data
   WHERE airport_code = p_airport_id;
   -- Если не найден вывод сообщения об ошибке
   IF v_airport_exists = 0 THEN
      RAISE_APPLICATION_ERROR(-20001, 'Ошибка: Аэропорт с ID ' || p_airport_id || ' не
найден.');
   END IF;
   airport_cursor := AirportManagementPackage.GetAirportHistory(p_airport_id,
ADD_MONTHS(SYSDATE, -120), SYSDATE);
   LOOP
      FETCH airport_cursor INTO v_flight_no, v_scheduled_departure, v_actual_departure,
v_arrival_airport, v_status;
      EXIT WHEN airport_cursor%NOTFOUND;
      v_total_flights := v_total_flights + 1; -- счетчик общего кол рейсов
      IF v_status = 'Cancelled' THEN
         v_cancelled_flights := v_cancelled_flights + 1; -- счетчик отмен
      END IF;

      IF v_actual_departure IS NOT NULL AND v_scheduled_departure IS NOT NULL
        AND v_actual_departure > v_scheduled_departure  THEN
         v_delayed_flights := v_delayed_flights + 1; -- счетчик задержек
      END IF;
   END LOOP;
   CLOSE airport_cursor;
   -- если аэропорт не существует или колич меньше 10
   IF v_total_flights < 10 THEN
```

```
      RAISE_APPLICATION_ERROR(-20002, 'Ошибка: Недостаточно данных для
анализа. Вылетов: ' || v_total_flights);
   END IF;

   DBMS_OUTPUT.PUT_LINE('Аэропорт: ' || p_airport_id);
   DBMS_OUTPUT.PUT_LINE('Всего вылетов: ' || v_total_flights);
   DBMS_OUTPUT.PUT_LINE('Отмененных: ' || v_cancelled_flights || ' (' ||
TO_CHAR(ROUND((v_cancelled_flights * 100.0) / v_total_flights, 2), '0.00') || '%)');
   DBMS_OUTPUT.PUT_LINE('Задержанных: ' || v_delayed_flights || ' (' ||
TO_CHAR(ROUND((v_delayed_flights * 100.0) / v_total_flights, 2), '00.00') || '%)');

DBMS_OUTPUT.PUT_LINE(CHR(10)||'Данные таблицы CANCEL_HISTORY_STATS
обновлены.');

    INSERT INTO cancel_history_stats (
     airport_id,
     total_flights, cancelled_flights, delayed_flights,
     cancelled_pct, delayed_pct
   ) VALUES (
     p_airport_id,
     v_total_flights, v_cancelled_flights, v_delayed_flights,
     ROUND(v_cancelled_flights*100/v_total_flights,2),
     ROUND(v_delayed_flights*100/v_total_flights,2)
   );
   COMMIT;

END CancelHistory;
-----------------------------------------------------------------

-- доп Среднее время полета и средняя задержка
FUNCTION CalculateFlightTime(
   p_departure_airport VARCHAR2,
   p_arrival_airport VARCHAR2
) RETURN VARCHAR2 IS
   v_avg_flight_seconds NUMBER;
   v_avg_delay_seconds NUMBER;
   v_result VARCHAR2(200);
BEGIN
--среднее время полета
   SELECT AVG(EXTRACT(DAY FROM (actual_arrival - actual_departure)) * 86400 +
          EXTRACT(HOUR FROM (actual_arrival - actual_departure)) * 3600 +
          EXTRACT(MINUTE FROM (actual_arrival - actual_departure)) * 60 +
          EXTRACT(SECOND FROM (actual_arrival - actual_departure)))
   INTO v_avg_flight_seconds
   FROM avia.flights
   WHERE departure_airport = p_departure_airport
     AND arrival_airport = p_arrival_airport
     AND actual_departure IS NOT NULL
```

```
    AND actual_arrival IS NOT NULL;
    -- среднняя задержка вылета
  SELECT AVG(EXTRACT(DAY FROM (actual_departure - scheduled_departure)) * 86400
+
          EXTRACT(HOUR FROM (actual_departure - scheduled_departure)) * 3600 +
          EXTRACT(MINUTE FROM (actual_departure - scheduled_departure)) * 60 +
          EXTRACT(SECOND FROM (actual_departure - scheduled_departure)))
  INTO v_avg_delay_seconds
  FROM avia.flights
  WHERE departure_airport = p_departure_airport
    AND arrival_airport = p_arrival_airport
    AND actual_departure IS NOT NULL
    AND scheduled_departure IS NOT NULL;
  v_result := 'Среднее время полета: ' || TRUNC(v_avg_flight_seconds / 3600) || ' ч ' ||
           TRUNC(MOD(v_avg_flight_seconds, 3600) / 60) || ' мин, ' ||
           'Средняя задержка вылета: ' || TRUNC(v_avg_delay_seconds / 3600) || ' ч ' ||
           TRUNC(MOD(v_avg_delay_seconds, 3600) / 60) || ' мин';
  RETURN v_result;
END CalculateFlightTime;
END AirportManagementPackage;
```

```
select * from avia.flights
WHERE departure_airport = 'DME'
AND arrival_airport = 'KZN';

select DISTINCT(departure_airport) from avia.flights;

select COUNT(*) from avia.flights
WHERE departure_airport = 'DME';

-- (доп)
SELECT AirportManagementPackage.CalculateFlightTime('DME', 'KZN')
FROM dual;

BEGIN
  AirportManagementPackage.CancelHistory('DME');
END;

BEGIN
  AirportManagementPackage.CancelHistory('MRV');
END;
```

```
select * from cancel_history_stats;



-- GetAirportHistory (id/плановый вылет/фактич вылет/)
DECLARE
   airport_report SYS_REFCURSOR;
   flight_no VARCHAR2(20);
   scheduled_departure DATE;
   actual_departure DATE;
   arrival_airport VARCHAR2(10);
   status VARCHAR2(20);
BEGIN
   airport_report := AirportManagementPackage.GetAirportHistory('SVO',
TO_DATE('2014-01-01', 'YYYY-MM-DD'), TO_DATE('2024-03-01', 'YYYY-MM-DD'));
   LOOP
      FETCH airport_report INTO flight_no, scheduled_departure, actual_departure,
arrival_airport, status;
      EXIT WHEN airport_report%NOTFOUND;
      DBMS_OUTPUT.PUT_LINE('Flight: ' || flight_no ||
                           ', Scheduled: ' || TO_CHAR(scheduled_departure, 'YYYY-MM-DD
HH24:MI') ||
                           ', Actual: ' || NVL(TO_CHAR(actual_departure, 'YYYY-MM-DD HH24:MI'),
'N/A') ||
                           ', Arrival: ' || arrival_airport ||
                           ', Status: ' || status);
   END LOOP;

   CLOSE airport_report;
END;


-- GenerateTravelReport (пассажиры/топ 3 направления/топ 3 с задержками)
DECLARE
   travel_report SYS_REFCURSOR;
   report_type VARCHAR2(100);
   report_value VARCHAR2(50);
BEGIN
   travel_report :=
AirportManagementPackage.GenerateTravelReport(TO_DATE('2015-01-01',
'YYYY-MM-DD'), TO_DATE('2024-03-01', 'YYYY-MM-DD'));

   LOOP
      FETCH travel_report INTO report_type, report_value;
      EXIT WHEN travel_report%NOTFOUND;
      DBMS_OUTPUT.PUT_LINE(report_type || ': ' || report_value);
```

```
    END LOOP;

    CLOSE travel_report;
END;

set SERVEROUTPUT on;
```

11 FlightBookingPackage

Функции:
Создайте в пакете "FlightBookingPackage"
функцию "IsSeatAvailable", которая принимает
параметры id пассажира, ID рейса и номер места и
возвращает флаг(0 или 1). Функция должна проверять
доступность места на рейсе.
Создайте функцию "GetPassengerBookings",
которая принимает id пассажира и возвращает
перечень всех его броней, при наличии.
Процедура с использованием функций и курсоров:
В пакете "FlightBookingPackage" создайте процедуру
"MakeBooking", которая принимает параметры: ID
пассажира, ID рейса и номер места. Процедура должна:
Проверять наличие пассажира и рейса.
• Использовать явные курсоры для выборки
информации о пассажире и рейсе.
Использовать функцию "IsSeatAvailable" для
проверки доступности места.
Если место доступно, добавить новую запись в
таблицу "Bookings".
. После окончания работы
добавленные записи необходимо аккуратно удалить. За
порчу исходной таблицы штраф (-1 балл за оставление
лишних записей после себя, -2 балла за удаления чего-
то кроме своих записей из исходных таблиц).
Если место занято или не доступно, выбросить
исключение с четким сообщением об ошибке.

select * from avia.seats; -- 773 37D 321 18E три вида класса комфорт, бизнес и эконом
select * from avia.aircrafts_data;
select * from avia.flights; -- 102 108
select * from avia.airports_data; -- 275 99
select * from avia.ticket_flights; -- 16157 4021 3660
select * from avia.tickets; -- 2290 450397 7066 161500 1115 708092 6903 458379 pass 0005432001061 0005432001066 0005432001062 tick D0B9CE EC1E8B
select * from avia.bookings;
select * from avia.boarding_passes; -- 1D 1C 16272 3660

```
create or replace package FlightBookingPackage is
function IsSeatAvailable(in_id in flights.flight_id%TYPE, in_seat in seats.seat_no%TYPE)
return number;
function GetPassengerBookings (pass_id tickets.passenger_id%TYPE) return varchar2;
procedure  MakeBooking ( Pass_id in varchar2, Reis_id in number, seat_num in varchar2);
function SummOfTicketsAndTotalCostOfFlight (in_id in flights.flight_id%TYPE ) return
varchar2;
function SummOfTicketsAndTotalCostOfFlightNEW(in_id in flights.flight_id%TYPE) return
varchar2;
end FlightBookingPackage;


CREATE OR REPLACE PACKAGE BODY FlightBookingPackage AS
   -- Тут ваша функция SummOfTicketsAndTotalCostOfFlight
END FlightBookingPackage;
/

set serveroutput on;

create or replace package body FlightBookingPackage is
function IsSeatAvailable(in_id in flights.flight_id%TYPE, in_seat in seats.seat_no%TYPE)
return number
is
   v_temp number;
   v_return number;
   v_max_seat_number number;
   v_max_seat_letter varchar(2);
begin
   select count(*) into v_temp
   from flights f, aircrafts_data a, seats s
   where f.aircraft_code=a.aircraft_code and  a.aircraft_code=s.aircraft_code
   and in_id = flight_id and in_seat = seat_no;

   select max(to_number(substr(seat_no, 1,2))) into v_max_seat_number
   from flights f, aircrafts_data a, seats s
   where  f.aircraft_code=a.aircraft_code and  a.aircraft_code=s.aircraft_code  and
length(seat_no) = 3 and in_id = flight_id;

   select max(substr(seat_no, -1)) into v_max_seat_letter
   from flights f, aircrafts_data a, seats s
   where  f.aircraft_code=a.aircraft_code and  a.aircraft_code=s.aircraft_code and in_id =
flight_id;

   if length(in_seat) = 2 then
      if substr(in_seat, -1) > v_max_seat_letter then
```

```
            DBMS_OUTPUT.PUT_LINE('Места ' ||  in_seat || ' на рейсе ' || in_id || ' не
существует.');
        else
            if v_temp > 0 then
                v_return:=0;
            else
                v_return:=1;
            end if;
       end if;
      else if length(in_seat) = 3 then
        if to_number(substr(in_seat, 1,2)) > v_max_seat_number and  substr(in_seat, -1) >
v_max_seat_letter then
            DBMS_OUTPUT.PUT_LINE('Места ' ||  in_seat || ' на рейсе ' || in_id || ' не
существует.');
        else
            if v_temp > 0 then
                v_return:=0;
            else
                v_return:=1;
        end if;
      end if;

    else
      DBMS_OUTPUT.PUT_LINE('Места ' || in_seat || ' на рейсе ' || in_id || ' не существует.');
    end if;
    end if;
    return v_return;
end IsSeatAvailable;

function GetPassengerBookings (pass_id tickets.passenger_id%TYPE) return varchar2
is
    G_vsego varchar2 (1024);
begin
    G_vsego:='';
    for rec in (
        select b.book_ref
        from tickets t, bookings b
        where b.book_ref = t.book_ref and t.passenger_id = pass_id)
    loop
        G_vsego := G_vsego||rec.book_ref||';';
    end loop;
    if length(G_vsego) > 0 then
        G_vsego := SUBSTR(G_vsego, 1, length(G_vsego) - 1);
    end if;
    return G_vsego;
end GetPassengerBookings;
```

```
procedure MakeBooking ( Pass_id in varchar2, Reis_id in number, seat_num in varchar2)
is
   v_book_ref tickets.book_ref%TYPE;
   v_total_amount ticket_flights.amount%TYPE;

   CURSOR passenger_cursor IS
     SELECT passenger_id
     FROM tickets
     WHERE passenger_id = Pass_id;

   passenger_rec tickets.passenger_id%TYPE;

   CURSOR flight_cursor IS
     SELECT flight_id
     FROM flights
     WHERE flight_id = Reis_id;

   flight_rec flights.flight_id%TYPE;

   seat_available NUMBER;

BEGIN
   OPEN passenger_cursor;
   FETCH passenger_cursor INTO passenger_rec;
   IF passenger_cursor%NOTFOUND THEN
     CLOSE passenger_cursor;
     RAISE_APPLICATION_ERROR(-20001, 'Пассажир не найден.');
   END IF;
   CLOSE passenger_cursor;

   OPEN flight_cursor;
   FETCH flight_cursor INTO flight_rec;
   IF flight_cursor%NOTFOUND THEN
     CLOSE flight_cursor;
     RAISE_APPLICATION_ERROR(-20002, 'Номер рейса не найдег.');
   END IF;
   CLOSE flight_cursor;

   seat_available := IsSeatAvailable(Reis_id, seat_num);

   if seat_available = 1
      then
      select t.book_ref, tf.amount into v_book_ref, v_total_amount
      from tickets t, ticket_flights tf
      where t.ticket_no=tf.ticket_no and tf.flight_id=Reis_id and rownum=1;

      INSERT INTO bookings2 (book_ref, book_date, total_amount)
      VALUES (v_book_ref, sysdate, v_total_amount);
```

```
        COMMIT;
        DBMS_OUTPUT.PUT_LINE('Бронирование успешно проведено для пассажира: ' ||
Pass_id || ', Номер рейса: ' || Reis_id || ', Посадочное место: ' || seat_num);
    else if seat_available = 0
        then
        DBMS_OUTPUT.PUT_LINE('Выбранное место уже занято');
        else
        DBMS_OUTPUT.PUT_LINE('Выбранного места не существует');
        end if;
    END IF;

EXCEPTION
    WHEN OTHERS THEN
      RAISE;
END MakeBooking;
-- разделить билеты на классы и на купленные и нет
function SummOfTicketsAndTotalCostOfFlight (in_id in flights.flight_id%TYPE ) return
varchar2
 is
 v_total_amount bookings.total_amount%TYPE;
 v_total_tickets number;
 v_vivod varchar2(128);
 v_klassC varchar2(40);
 v_klassB varchar2(40);
 v_klassE varchar2(40);
 v_tickets_no number;
 begin

    select sum(b.total_amount), count(*) into v_total_amount, v_total_tickets
    from bookings b
    join tickets t on b.book_ref = t.book_ref
    join ticket_flights tf on t.ticket_no = tf.ticket_no
    join ticket_flights tf on t.fare_conditions = tf.ticket_no
    where tf.flight_id=in_id;

    if v_total_amount is Null or v_total_tickets is Null
    then
        v_vivod:=('Такого рейса не существует или по нему нет продаж');
    else
        v_vivod:=('Сумма: ' || v_total_amount || ', Количество билетов: ' || v_total_tickets || ',
Количество билетов класса Comfort: ' || v_klassC || ', Количество билетов класса
Business: ' || v_klassB || ', Количество билетов класса Economy: ' || v_klassE || ',
Количество билетов, которые не продали: ' ||  v_tickets_no ||);
    end if;
    return v_vivod;
end SummOfTicketsAndTotalCostOfFlight;
```

```
CREATE OR REPLACE FUNCTION SummOfTicketsAndTotalCostOfFlightNEW(in_id IN
flights.flight_id%TYPE)
   RETURN VARCHAR2 IS
   v_vivod VARCHAR2(4000);
   v_found BOOLEAN := FALSE;

   CURSOR ticket_cursor IS
      SELECT
         tf.fare_conditions AS fare_class,
         SUM(CASE WHEN b.book_ref IS NOT NULL THEN tf.amount ELSE 0 END) AS
total_sold_amount,
         COUNT(CASE WHEN b.book_ref IS NOT NULL THEN 1 END) AS
sold_tickets_count,
         COUNT(CASE WHEN b.book_ref IS NULL THEN 1 END) AS unsold_tickets_count
      FROM avia.ticket_flights tf
      LEFT JOIN avia.tickets t ON tf.ticket_no = t.ticket_no
      LEFT JOIN avia.bookings b ON t.book_ref = b.book_ref
      WHERE tf.flight_id = in_id
      GROUP BY tf.fare_conditions;

   v_fare_class VARCHAR2(30);
   v_total_sold_amount NUMBER;
   v_sold_tickets_count NUMBER;
   v_unsold_tickets_count NUMBER;
BEGIN
   v_vivod := 'Информация о рейсе ' || in_id || ':' || CHR(10);

   FOR rec IN ticket_cursor LOOP
      v_found := TRUE;

      v_fare_class := rec.fare_class;
      v_total_sold_amount := rec.total_sold_amount;
      v_sold_tickets_count := rec.sold_tickets_count;
      v_unsold_tickets_count := rec.unsold_tickets_count;

      v_vivod := v_vivod ||
               'Класс: ' || v_fare_class || CHR(10) ||
               '  Сумма продаж: ' || NVL(v_total_sold_amount, 0) || CHR(10) ||
               '  Куплено билетов: ' || NVL(v_sold_tickets_count, 0) || CHR(10) ||
               '  Не куплено билетов: ' || NVL(v_unsold_tickets_count, 0) || CHR(10);
   END LOOP;

   IF NOT v_found THEN
      v_vivod := 'Такого рейса не существует или по нему нет продаж.';
   END IF;

   RETURN v_vivod;
END SummOfTicketsAndTotalCostOfFlightNEW;
```

```
end FlightBookingPackage; --конец!!!!!!!


DECLARE
   result VARCHAR2(4000);
BEGIN
   result := SummOfTicketsAndTotalCostOfFlightNEW(30625); -- Укажите ID рейса
   DBMS_OUTPUT.PUT_LINE(result);
END;


select * from tickets;
select * from boarding_passes;




DECLARE
   v_result VARCHAR2(4000);
BEGIN
   v_result := SummOfTicketsAndTotalCostOfFlightNEW(30625);
   dbms_output.put_line(v_result);
END;



select FlightBookingPackage.SummOfTicketsAndTotalCostOfFlight(16272)
from dual;

select FlightBookingPackage.GetPassengerBookings('8141 604011')
from dual;

select FlightBookingPackage.FlightBookingPackage;

set serveroutput on;

declare
   result number;
begin
   result := flightbookingpackage.isseatavailable(16272, '15A');
   dbms_output.put_line('результат: ' || result);
end;

BEGIN
   FlightBookingPackage.MakeBooking('2290 450397', 30625, '99Q');
END;
```

```
declare
   bookings_list varchar2(1024);
begin
   bookings_list := flightbookingpackage.getpassengerbookings('7922 672476');
   dbms_output.put_line('бронирования: ' || bookings_list);
end;



begin
   flightbookingpackage.makebooking('2290 450397', 0005432001061, '16A');
end;


declare
   result varchar2(128);
begin
   result := flightbookingpackage.summofticketsandtotalcostofflight(16157);
   dbms_output.put_line('результат: ' || result);
end;
```

12 BookingManagementPackage

```
CREATE OR REPLACE PACKAGE BookingManagementPackage AUTHID DEFINER AS

        -- Функция для получения доступных рейсов

        FUNCTION GetAvailableFlights RETURN SYS_REFCURSOR;



        -- Функция для получения деталей о рейсе по ID

        FUNCTION GetFlightDetails(flight_id IN Test_Flights.FLIGHT_ID%TYPE)

        RETURN SYS_REFCURSOR;



        -- Измененная функция для расчета общей стоимости билетов по модели
самолета

        FUNCTION CalculateTotalFlightRevenue(aircraft_model IN
Test_Aircrafts.model%TYPE)

        RETURN NUMBER;
```

```
	-- Процедура для отмены бронирования
	PROCEDURE CancelBooking(
	p_book_ref IN Test_Bookings.book_ref%TYPE
	);
END BookingManagementPackage;
/
```

```
CREATE OR REPLACE PACKAGE BODY BookingManagementPackage AS
  -- Функция для расчета общей стоимости билетов по модели самолета
	FUNCTION CalculateTotalFlightRevenue(aircraft_model IN
Test_Aircrafts.model%TYPE)
	RETURN NUMBER IS
	v_total_revenue NUMBER := 0;
	BEGIN
	-- Считаем общую стоимость всех билетов для указанной модели самолета
	SELECT NVL(SUM(tf.amount), 0)
	INTO v_total_revenue
	FROM Test_Ticket_flights tf
	JOIN Test_Flights f ON tf.flight_id = f.flight_id
	JOIN Test_Aircrafts a ON f.aircraft_code = a.aircraft_code
	WHERE a.model = aircraft_model;

	RETURN v_total_revenue;
	EXCEPTION
```

```
        WHEN NO_DATA_FOUND THEN

        RAISE_APPLICATION_ERROR(-20011, 'Данные для указанной модели самолета не найдены.');

        WHEN OTHERS THEN

        RAISE_APPLICATION_ERROR(-20012, 'Ошибка при расчете общей стоимости билетов: ' || SQLERRM);

        END CalculateTotalFlightRevenue;

        -- Функция для получения доступных рейсов

FUNCTION GetAvailableFlights RETURN SYS_REFCURSOR IS

        v_cursor SYS_REFCURSOR;

        v_flight_id Test_Flights.FLIGHT_ID%TYPE;

        v_flight_no Test_Flights.FLIGHT_NO%TYPE;

        v_scheduled_departure Test_Flights.SCHEDULED_DEPARTURE%TYPE;

        v_scheduled_arrival Test_Flights.SCHEDULED_ARRIVAL%TYPE;

        v_status Test_Flights.STATUS%TYPE;

        v_available_seats NUMBER;

BEGIN

        OPEN v_cursor FOR

        SELECT

        f.flight_id,

        f.flight_no,

            f.scheduled_departure,

        f.scheduled_arrival,

        f.status,

        f.seats_total - NVL(COUNT(tf.ticket_no), 0) AS available_seats

        FROM Test_Flights f

        LEFT JOIN Test_Ticket_flights tf ON f.flight_id = tf.flight_id
```

```
WHERE f.status NOT IN ('Cancelled', 'Departed')

GROUP BY f.flight_id, f.flight_no, f.scheduled_departure, f.scheduled_arrival,
f.status, f.seats_total

HAVING (f.seats_total - NVL(COUNT(tf.ticket_no), 0)) > 0;


-- Вывод данных без закрытия курсора

DBMS_OUTPUT.PUT_LINE('Available Flights:');

LOOP

FETCH v_cursor INTO

v_flight_id,

v_flight_no,

v_scheduled_departure,

v_scheduled_arrival,

v_status,

v_available_seats;

EXIT WHEN v_cursor%NOTFOUND;


DBMS_OUTPUT.PUT_LINE(

'Flight ID: ' || v_flight_id ||

', Flight No: ' || v_flight_no ||

', Available Seats: ' || v_available_seats

);

END LOOP;


-- Возвращаем курсор в исходное положение

IF v_cursor%ISOPEN THEN

CLOSE v_cursor;
```

```
        END IF;

        -- Повторно открываем курсор для внешнего использования
        OPEN v_cursor FOR
        SELECT
        f.flight_id,
        f.flight_no,
        f.scheduled_departure,
        f.scheduled_arrival,
        f.status,
        f.seats_total - NVL(COUNT(tf.ticket_no), 0) AS available_seats
        FROM Test_Flights f
        LEFT JOIN Test_Ticket_flights tf ON f.flight_id = tf.flight_id
        WHERE f.status NOT IN ('Cancelled', 'Departed')
        GROUP BY f.flight_id, f.flight_no, f.scheduled_departure, f.scheduled_arrival,
f.status, f.seats_total
        HAVING (f.seats_total - NVL(COUNT(tf.ticket_no), 0)) > 0;

        RETURN v_cursor;
END GetAvailableFlights;

        -- Функция для получения деталей о рейсе по ID
        -- Функция для получения деталей о рейсе по ID
FUNCTION GetFlightDetails(flight_id IN Test_Flights.FLIGHT_ID%TYPE)
RETURN SYS_REFCURSOR IS
        v_cursor SYS_REFCURSOR;
        v_scheduled_departure Test_Flights.SCHEDULED_DEPARTURE%TYPE;
```

```
        v_scheduled_arrival Test_Flights.SCHEDULED_ARRIVAL%TYPE;
        v_departure_airport Test_Flights.DEPARTURE_AIRPORT%TYPE;
        v_arrival_airport Test_Flights.ARRIVAL_AIRPORT%TYPE;
        v_aircraft_code Test_Flights.AIRCRAFT_CODE%TYPE;
        v_actual_departure Test_Flights.ACTUAL_DEPARTURE%TYPE;
        v_actual_arrival Test_Flights.ACTUAL_ARRIVAL%TYPE;
BEGIN
        -- Открываем курсор для деталей рейса
        OPEN v_cursor FOR
        SELECT
        scheduled_departure,
        scheduled_arrival,
        departure_airport,
        arrival_airport,
        aircraft_code,
        actual_departure,
        actual_arrival
        FROM Test_Flights
        WHERE flight_id = GetFlightDetails.flight_id;

        -- Вывод данных о рейсе
        DBMS_OUTPUT.PUT_LINE('Flight Details for Flight ID: ' || flight_id);
        LOOP
        FETCH v_cursor INTO
        v_scheduled_departure,
        v_scheduled_arrival,
```

```
v_departure_airport,

v_arrival_airport,

v_aircraft_code,

v_actual_departure,

v_actual_arrival;

EXIT WHEN v_cursor%NOTFOUND;


DBMS_OUTPUT.PUT_LINE(

'Scheduled Departure: ' || TO_CHAR(v_scheduled_departure, 'YYYY-MM-DD
HH24:MI:SS') ||

', Scheduled Arrival: ' || TO_CHAR(v_scheduled_arrival, 'YYYY-MM-DD
HH24:MI:SS') ||

', Departure Airport: ' || v_departure_airport ||

', Arrival Airport: ' || v_arrival_airport ||

', Aircraft Code: ' || v_aircraft_code ||

', Actual Departure: ' || NVL(TO_CHAR(v_actual_departure, 'YYYY-MM-DD
HH24:MI:SS'), 'N/A') ||

', Actual Arrival: ' || NVL(TO_CHAR(v_actual_arrival, 'YYYY-MM-DD HH24:MI:SS'),
'N/A')

);

END LOOP;


-- Закрываем курсор после вывода

IF v_cursor%ISOPEN THEN

CLOSE v_cursor;

END IF;


-- Повторно открываем курсор для внешнего использования
```

```
        OPEN v_cursor FOR
        SELECT
        scheduled_departure,
        scheduled_arrival,
        departure_airport,
        arrival_airport,
        aircraft_code,
        actual_departure,
        actual_arrival
        FROM Test_Flights
        WHERE flight_id = GetFlightDetails.flight_id;

        RETURN v_cursor;
END GetFlightDetails;
        -- Функция для проверки доступности места на рейсе
        -- Функция для расчета общей стоимости билетов на рейс



        -- Процедура для отмены бронирования
        PROCEDURE CancelBooking(p_book_ref IN Test_Bookings.book_ref%TYPE) IS
        -- Переменные для хранения данных бронирования
        v_total_amount Test_Bookings.total_amount%TYPE;
        v_book_date Test_Bookings.book_date%TYPE;

        -- Курсор для получения данных билетов
        CURSOR c_tickets(p_book_ref IN Test_Bookings.book_ref%TYPE) IS
```

```
        SELECT t.ticket_no, tf.flight_id, t.passenger_id, t.passenger_name, t.contact_data,
tf.fare_conditions, tf.amount

        FROM Test_Tickets t

        JOIN Test_Ticket_flights tf ON t.ticket_no = tf.ticket_no

        WHERE t.book_ref = p_book_ref;


        -- Переменные для хранения деталей рейса

    v_scheduled_departure Test_Flights.scheduled_departure%TYPE;

        v_departure_airport Test_Flights.departure_airport%TYPE;

        v_arrival_airport Test_Flights.arrival_airport%TYPE;

        v_aircraft_code Test_Flights.aircraft_code%TYPE;

        v_actual_departure Test_Flights.actual_departure%TYPE;

        v_actual_arrival Test_Flights.actual_arrival%TYPE;


        -- Переменная для текущей даты и времени

        v_current_date DATE := SYSDATE;


        -- Исключения

        e_booking_not_found EXCEPTION;

        e_cancellation_too_late EXCEPTION;


        -- Переменная для работы с курсором, возвращаемым функцией
GetFlightDetails

        v_flight_cursor SYS_REFCURSOR;

BEGIN

        -- Проверка существования бронирования

        BEGIN
```

```
SELECT b.total_amount, b.book_date

INTO v_total_amount, v_book_date

FROM Test_Bookings b

WHERE b.book_ref = p_book_ref;

EXCEPTION

WHEN NO_DATA_FOUND THEN

RAISE_APPLICATION_ERROR(-20001, 'Бронирование с указанным book_ref не найдено');

END;


-- Обработка билетов

FOR rec_ticket IN c_tickets(p_book_ref) LOOP

DBMS_OUTPUT.PUT_LINE('Processing ticket: ' || rec_ticket.ticket_no);


-- Получение деталей рейса через функцию GetFlightDetails

BEGIN

v_flight_cursor := GetFlightDetails(rec_ticket.flight_id);


-- Проверка, открыт ли курсор

IF NOT v_flight_cursor%ISOPEN THEN

    RAISE_APPLICATION_ERROR(-20006, 'Курсор не был открыт.');

END IF;


-- Извлечение данных из курсора

FETCH v_flight_cursor INTO

    v_scheduled_departure,

    v_departure_airport,
```

```
            v_arrival_airport,
            v_aircraft_code,
            v_actual_departure,
            v_actual_arrival;

        -- Проверка, были ли извлечены данные
        IF v_flight_cursor%NOTFOUND THEN
            CLOSE v_flight_cursor;
            RAISE_APPLICATION_ERROR(-20007, 'Данные о рейсе не найдены.');
        END IF;

        -- Закрытие курсора
        CLOSE v_flight_cursor;

        EXCEPTION
        WHEN OTHERS THEN
            IF v_flight_cursor%ISOPEN THEN
            CLOSE v_flight_cursor;
            END IF;
            RAISE_APPLICATION_ERROR(-20008, 'Ошибка при получении деталей
рейса: ' || SQLERRM);
        END;

        -- Проверка времени до вылета
        IF v_scheduled_departure - v_current_date < 1 THEN
        RAISE e_cancellation_too_late;
        END IF;
```

```
-- Сохранение данных билетов в промежуточную таблицу
INSERT INTO CancelledBookings (
book_ref,
ticket_no,
flight_id,
passenger_id,
passenger_name,
contact_data,
fare_conditions,
amount,
scheduled_departure,
  departure_airport,
arrival_airport,
aircraft_code,
total_amount,
book_date
)
VALUES (
p_book_ref,
rec_ticket.ticket_no,
rec_ticket.flight_id,
rec_ticket.passenger_id,
rec_ticket.passenger_name,
rec_ticket.contact_data,
rec_ticket.fare_conditions,
```

```
        rec_ticket.amount,
        v_scheduled_departure,
        v_departure_airport,
        v_arrival_airport,
        v_aircraft_code,
        v_total_amount,
        v_book_date
        );
        END LOOP;

        -- Удаление данных из исходных таблиц
        DELETE FROM Test_Ticket_flights WHERE ticket_no IN (
        SELECT ticket_no FROM Test_Tickets WHERE book_ref = p_book_ref
        );
        DELETE FROM Test_Tickets WHERE book_ref = p_book_ref;
        DELETE FROM Test_Bookings WHERE book_ref = p_book_ref;

        COMMIT;
EXCEPTION
        WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20001, 'Бронирование с указанным book_ref не найдено');
        WHEN e_cancellation_too_late THEN
        RAISE_APPLICATION_ERROR(-20002, 'Отмена бронирования невозможна: до вылета рейса осталось менее суток');
        WHEN OTHERS THEN
        ROLLBACK;
```

```
	RAISE_APPLICATION_ERROR(-20003, 'Произошла ошибка при отмене бронирования: ' || SQLERRM);

END CancelBooking;



END BookingManagementPackage;

/

select * from test_ticket_flights



select * from test_flights



select * from test_aircrafts



DECLARE

	v_total_revenue NUMBER;

	v_aircraft_model VARCHAR2(100) := 'Boeing 747';

BEGIN

	v_total_revenue := BookingManagementPackage.CalculateTotalFlightRevenue(v_aircraft_model);

   DBMS_OUTPUT.PUT_LINE('Общая стоимость билетов для модели ' || v_aircraft_model || ': ' || TO_CHAR(v_total_revenue));

END;

/
```

13 FlightBookingPackage

```
create or replace package FlightBookingPackage is
function IsSeatAvailable(in_id in flights.flight_id%TYPE, in_seat in seats.seat_no%TYPE) return number;
function GetPassengerBookings (pass_id tickets.passenger_id%TYPE) return varchar2;
procedure  MakeBooking ( Pass_id in varchar2, Reis_id in number, seat_num in varchar2);
function SummOfTicketsAndTotalCostOfFlight (in_id in flights.flight_id%TYPE ) return varchar2;
```

```
end FlightBookingPackage;

create or replace package body FlightBookingPackage is
function IsSeatAvailable(in_id in flights.flight_id%TYPE, in_seat in seats.seat_no%TYPE)
return number
is
   v_temp number;
   v_return number;
   v_max_seat_number number;
   v_max_seat_letter varchar(2);
begin
   select count(*) into v_temp
   from flights f, aircrafts_data a, seats s
   where f.aircraft_code=a.aircraft_code and  a.aircraft_code=s.aircraft_code
   and in_id = flight_id and in_seat = seat_no;

   select max(to_number(substr(seat_no, 1,2))) into v_max_seat_number
   from flights f, aircrafts_data a, seats s
   where  f.aircraft_code=a.aircraft_code and  a.aircraft_code=s.aircraft_code  and
length(seat_no) = 3 and in_id = flight_id;

   select max(substr(seat_no, -1)) into v_max_seat_letter
   from flights f, aircrafts_data a, seats s
   where  f.aircraft_code=a.aircraft_code and  a.aircraft_code=s.aircraft_code and in_id =
flight_id;

   if length(in_seat) = 2 then
      if substr(in_seat, -1) > v_max_seat_letter then
          DBMS_OUTPUT.PUT_LINE('Места ' ||  in_seat || ' на рейсе ' || in_id || ' не
существует.');
      else
          if v_temp > 0 then
             v_return:=0;
          else
             v_return:=1;
          end if;
     end if;
    else if length(in_seat) = 3 then
       if to_number(substr(in_seat, 1,2)) > v_max_seat_number and  substr(in_seat, -1) >
v_max_seat_letter then
          DBMS_OUTPUT.PUT_LINE('Места ' ||  in_seat || ' на рейсе ' || in_id || ' не
существует.');
      else
          if v_temp > 0 then
             v_return:=0;
          else
             v_return:=1;
      end if;
```

```
     end if;

    else
      DBMS_OUTPUT.PUT_LINE('Места ' || in_seat || ' на рейсе ' || in_id || ' не существует.');
    end if;
    end if;
    return v_return;
end IsSeatAvailable;

function GetPassengerBookings (pass_id tickets.passenger_id%TYPE) return varchar2
is
    G_vsego varchar2 (1024);
begin
    G_vsego:='';
    for rec in (
        select b.book_ref
        from tickets t, bookings b
        where b.book_ref = t.book_ref and t.passenger_id = pass_id)
    loop
        G_vsego := G_vsego||rec.book_ref||';';
    end loop;
    if length(G_vsego) > 0 then
        G_vsego := SUBSTR(G_vsego, 1, length(G_vsego) - 1);
    end if;
    return G_vsego;
end GetPassengerBookings;

procedure MakeBooking ( Pass_id in varchar2, Reis_id in number, seat_num in varchar2)
is
    v_book_ref tickets.book_ref%TYPE;
    v_total_amount ticket_flights.amount%TYPE;

    CURSOR passenger_cursor IS
      SELECT passenger_id
      FROM tickets
      WHERE passenger_id = Pass_id;

    passenger_rec tickets.passenger_id%TYPE;

    CURSOR flight_cursor IS
      SELECT flight_id
      FROM flights
      WHERE flight_id = Reis_id;

    flight_rec flights.flight_id%TYPE;

    seat_available NUMBER;
```

```
BEGIN
   OPEN passenger_cursor;
   FETCH passenger_cursor INTO passenger_rec;
   IF passenger_cursor%NOTFOUND THEN
     CLOSE passenger_cursor;
     RAISE_APPLICATION_ERROR(-20001, 'Пассажир не найден.');
   END IF;
   CLOSE passenger_cursor;

   OPEN flight_cursor;
   FETCH flight_cursor INTO flight_rec;
   IF flight_cursor%NOTFOUND THEN
     CLOSE flight_cursor;
     RAISE_APPLICATION_ERROR(-20002, 'Номер рейса не найдег.');
   END IF;
   CLOSE flight_cursor;

   seat_available := IsSeatAvailable(Reis_id, seat_num);

   if seat_available = 1
      then
      select t.book_ref, tf.amount into v_book_ref, v_total_amount
      from tickets t, ticket_flights tf
      where t.ticket_no=tf.ticket_no and tf.flight_id=Reis_id and rownum=1;

      INSERT INTO bookings2 (book_ref, book_date, total_amount)
      VALUES (v_book_ref, sysdate, v_total_amount);
      COMMIT;
      DBMS_OUTPUT.PUT_LINE('Бронирование успешно проведено для пассажира: ' ||
Pass_id ||
                        ', Номер рейса: ' || Reis_id || ', Посадочное место: ' || seat_num);
   else if seat_available = 0
      then
      DBMS_OUTPUT.PUT_LINE('Выбранное место уже занято');
      else
      DBMS_OUTPUT.PUT_LINE('Выбранного места не существует');
      end if;
   END IF;

EXCEPTION
   WHEN OTHERS THEN
     RAISE;
END MakeBooking;

function SummOfTicketsAndTotalCostOfFlight (in_id in flights.flight_id%TYPE ) return
varchar2
 is
 v_total_amount bookings.total_amount%TYPE;
```

```
 v_total_tickets number;
 v_vivod varchar2(128);
 begin

    select sum(b.total_amount), count(*) into v_total_amount, v_total_tickets
    from bookings b
    join tickets t on b.book_ref = t.book_ref
    join ticket_flights tf on t.ticket_no = tf.ticket_no
    where tf.flight_id=in_id;

    if v_total_amount is Null or v_total_tickets is Null
    then
        v_vivod:=('Такого рейса не существует или по нему нет продаж');
    else
        v_vivod:=('Сумма: ' || v_total_amount || ', Количество билетов: ' || v_total_tickets);
    end if;
    return v_vivod;
end SummOfTicketsAndTotalCostOfFlight;


end FlightBookingPackage;
select * from tickets;
select * from boarding_passes;


begin
    FlightBookingPackage.Makebooking('8149 604011', 30625, '99Q');
end;

select FlightBookingPackage.SummOfTicketsAndTotalCostOfFlight(30625)
from dual;

select FlightBookingPackage.GetPassengerBookings('8141 604011')
from dual;

select FlightBookingPackage.FlightBookingPackage;

select * from bookings2;
select * from bookings3;
create table bookings2 as (select * from bookings where rownum <1);
```

14 FlightManagementPackage

Создание пакета:
Функции:
Создайте в пакете «FlightManagementPackage» функцию
*GetAvailableFlights", которая принимает на вход интервал дат и

аэропорт вылета. Эта функция должна возвращать список доступных в указанном интервале времени рейсов, вылетающих из указанного аэропорта. Рейсы считаются доступными, если у них есть свободные места и они еще не вылетели или не отменены.
Создайте в пакете «FlightManagementPackage» функцию *GetDistanceKm". Эта функция должна принимать ID аэропорта вылета и прилета и возвращать расстояние между ними в километрах.
Процедура с использованием функций и курсоров:
Эта процедура должна принимать информацию о новом рейсе и добавлять его в систему. Процедура должна:
Проверять отсутствие указанного рейса в базе данных.
Проверьте, что максимальная дальность самолета, выбранного для осуществления рейса, превышает расстояние между аэропортами хот бы в 1,2 раза.
Использовать функцию "GetDistanceKm" для расчета расстояния между аэропортами.
Использовать явные курсоры для выборки информации о рейсах.
Если указанный рейс уже существует или максимальная дальность полета самолета не подходи для выполнения указанного рейса, выбросить исключение с четким сообщением об ошибке.
После завершения работы все добавленные записи необходимо удалить. За порчу исходной таблицы штраф (-1 балл за оставление лишних записей после себя, -2 балла за удаления чего-то кроме своих записей из исходных таблиц).
Дополнительное задание:
Придумайте и реализуйте в пакете свою функцию. Функция должна быть максимально практически полезна для указанной предметной области.
Если функция повторяется функционал, придуманный другим студентом, то оценка за нее будет снижена на бал, и она должна быть реализована иначе, чем у сделавших ее ранее.

```
--создаём временную таблицу чтобы не затереть исходные
create table my_flights as select * from avia.flights;


--функция создания инкремента
DECLARE
  v_new_start NUMBER;
BEGIN

  SELECT NVL(MAX(flight_id), 0) + 1 INTO v_new_start FROM my_flights;


  BEGIN
    EXECUTE IMMEDIATE 'DROP SEQUENCE flights_seq';
  EXCEPTION
```

```
    WHEN OTHERS THEN
      NULL;
  END;


  EXECUTE IMMEDIATE 'CREATE SEQUENCE flights_seq START WITH ' || v_new_start ||
                ' INCREMENT BY 1 NOCACHE NOCYCLE';

  DBMS_OUTPUT.PUT_LINE('Создан инкремент flights_seq, начальное значение ' ||
v_new_start);
END;
/

set serveroutput on;

--создание пакета FlightManagementPackage
CREATE OR REPLACE PACKAGE FlightManagementPackage AUTHID DEFINER AS
    TYPE flight_refcursor IS REF CURSOR;

    FUNCTION GetAvailableFlights(
        start_date DATE,
        end_date DATE,
        departure_airport_code VARCHAR2
    ) RETURN flight_refcursor;

    FUNCTION GetDistanceKm(
        airport_code1 VARCHAR2,
        airport_code2 VARCHAR2
    ) RETURN NUMBER;

    PROCEDURE AddNewFlight(
        p_flight_no             IN avia.flights.flight_no%TYPE,
        p_scheduled_departure   IN avia.flights.scheduled_departure%TYPE,
        p_scheduled_arrival     IN avia.flights.scheduled_arrival%TYPE,
        p_departure_airport     IN avia.flights.departure_airport%TYPE,
        p_arrival_airport       IN avia.flights.arrival_airport%TYPE,
        p_status                IN avia.flights.status%TYPE,
        p_aircraft_code         IN avia.flights.aircraft_code%TYPE
    );

    FUNCTION MyFunction(
        p_flight_no   IN VARCHAR2,
        p_start_date  IN DATE,
        p_end_date    IN DATE
    ) RETURN VARCHAR2;
END FlightManagementPackage;
/
```

```
--создание тела пакета FlightManagementPackage
CREATE OR REPLACE PACKAGE BODY FlightManagementPackage AS

   FUNCTION GetAvailableFlights(
      start_date DATE,
      end_date DATE,
      departure_airport_code VARCHAR2
   ) RETURN flight_refcursor IS
      flights flight_refcursor;
   BEGIN
      OPEN flights FOR
         SELECT f.flight_id,
               f.flight_no,
               f.scheduled_departure,
               f.departure_airport,
               f.status
         FROM avia.flights f
         WHERE f.departure_airport = departure_airport_code
           AND f.scheduled_departure >= start_date
           AND f.scheduled_departure < end_date + 1
           AND f.status IN ('Scheduled')
           AND EXISTS (
                SELECT 1
                FROM avia.seats s
                JOIN avia.flights f2 ON s.aircraft_code = f2.aircraft_code
                WHERE f2.flight_id = f.flight_id
                  AND s.seat_no NOT IN (
                      SELECT seat_no
                      FROM avia.boarding_passes
                      WHERE flight_id = f.flight_id
                  )
           );
      RETURN flights;
   END GetAvailableFlights;

   FUNCTION GetDistanceKm(
      airport_code1 VARCHAR2,
      airport_code2 VARCHAR2
   ) RETURN NUMBER IS
      lon1 NUMBER;
      lat1 NUMBER;
      lon2 NUMBER;
      lat2 NUMBER;
      dist NUMBER;
      pi CONSTANT NUMBER := ACOS(-1);
   BEGIN
```

```
        EXECUTE IMMEDIATE 'ALTER SESSION SET NLS_NUMERIC_CHARACTERS =
'.,'';
        SELECT
            TO_NUMBER(REGEXP_SUBSTR(coordinates, '[-0-9.]+', 1, 1)),
            TO_NUMBER(REGEXP_SUBSTR(coordinates, '[-0-9.]+', 1, 2))
        INTO lon1, lat1
        FROM avia.airports_data
        WHERE airport_code = airport_code1;

        SELECT
            TO_NUMBER(REGEXP_SUBSTR(coordinates, '[-0-9.]+', 1, 1)),
            TO_NUMBER(REGEXP_SUBSTR(coordinates, '[-0-9.]+', 1, 2))
        INTO lon2, lat2
        FROM avia.airports_data
        WHERE airport_code = airport_code2;

        dist := 6371 * ACOS(
            COS(lat1 * pi / 180) * COS(lat2 * pi / 180) *
            COS((lon2 - lon1) * pi / 180) +
            SIN(lat1 * pi / 180) * SIN(lat2 * pi / 180)
        );
        RETURN dist;
    END GetDistanceKm;

    PROCEDURE AddNewFlight(
        p_flight_no             IN avia.flights.flight_no%TYPE,
        p_scheduled_departure   IN avia.flights.scheduled_departure%TYPE,
        p_scheduled_arrival     IN avia.flights.scheduled_arrival%TYPE,
        p_departure_airport     IN avia.flights.departure_airport%TYPE,
        p_arrival_airport       IN avia.flights.arrival_airport%TYPE,
        p_status                IN avia.flights.status%TYPE,
        p_aircraft_code         IN avia.flights.aircraft_code%TYPE
    ) IS
        v_flight_count NUMBER;
        v_max_range NUMBER;
        v_distance NUMBER;
        v_new_flight_id NUMBER;
        v_flight_no my_flights.flight_no%TYPE;

CURSOR cur_flight_exists IS
    SELECT flight_no
    FROM my_flights
    WHERE flight_no = p_flight_no
      AND scheduled_departure = p_scheduled_departure
      AND departure_airport = p_departure_airport
      AND arrival_airport = p_arrival_airport;

BEGIN
```

```
    v_flight_count := 0;
    OPEN cur_flight_exists;
    LOOP
        FETCH cur_flight_exists INTO v_flight_no;
        EXIT WHEN cur_flight_exists%NOTFOUND;
        v_flight_count := v_flight_count + 1;
    END LOOP;
    CLOSE cur_flight_exists;

    IF v_flight_count > 0 THEN
        DBMS_OUTPUT.PUT_LINE('Ошибка: Рейс уже существует.');
        RETURN;
    END IF;
        SELECT "RANGE"
          INTO v_max_range
          FROM avia.aircrafts_data
         WHERE aircraft_code = p_aircraft_code;

        v_distance := FlightManagementPackage.GetDistanceKm(p_departure_airport,
p_arrival_airport);

        IF v_max_range < 1.2 * v_distance THEN
            DBMS_OUTPUT.PUT_LINE('Ошибка: Дальность полёта самолёта
недостаточна.');
            RETURN;
        END IF;

        INSERT INTO my_flights (
            flight_id,
            flight_no,
            scheduled_departure,
            scheduled_arrival,
            departure_airport,
            arrival_airport,
            status,
            aircraft_code
        ) VALUES (
            flights_seq.NEXTVAL,
            p_flight_no,
            p_scheduled_departure,
            p_scheduled_arrival,
            p_departure_airport,
            p_arrival_airport,
            p_status,
            p_aircraft_code
        )
        RETURNING flight_id INTO v_new_flight_id;
```

```
      DBMS_OUTPUT.PUT_LINE('Рейс успешно добавлен. Flight ID: ' || v_new_flight_id);

   EXCEPTION
      WHEN OTHERS THEN
         DBMS_OUTPUT.PUT_LINE('Ошибка: ' || SQLERRM);
         RETURN;
   END AddNewFlight;

   FUNCTION MyFunction(
      p_flight_no   IN VARCHAR2,
      p_start_date  IN DATE,
      p_end_date    IN DATE
   ) RETURN VARCHAR2 IS
      v_revenue NUMBER;
   v_dep_airport avia.flights.departure_airport%TYPE;
   v_arr_airport avia.flights.arrival_airport%TYPE;
   v_result      VARCHAR2(256);
   BEGIN
      SELECT NVL(SUM(tf.amount), 0)
        INTO v_revenue
        FROM avia.ticket_flights tf
        JOIN avia.flights f ON tf.flight_id = f.flight_id
        JOIN avia.tickets t ON tf.ticket_no = t.ticket_no
        JOIN avia.bookings b ON t.book_ref = b.book_ref
       WHERE f.flight_no = p_flight_no
         AND b.book_date BETWEEN p_start_date AND p_end_date;

   SELECT departure_airport, arrival_airport
     INTO v_dep_airport, v_arr_airport
     FROM avia.flights
     WHERE flight_no = p_flight_no
       AND ROWNUM = 1;
         v_result :=   TO_CHAR(v_revenue) ||','||p_flight_no  ||','|| v_dep_airport ||','||
v_arr_airport;
      RETURN v_result;
   EXCEPTION
      WHEN OTHERS THEN
         RETURN 0;
   END MyFunction;

END FlightManagementPackage;
/

--пример использования функции GetAvailableFlights
VAR flights REFCURSOR;

BEGIN
   :flights := FlightManagementPackage.GetAvailableFlights(
```

```
        TO_DATE('2017-09-05', 'YYYY-MM-DD'),
        TO_DATE('2017-09-09', 'YYYY-MM-DD'),
        'SVO'
    );
END;
/
PRINT flights;


--пример использования функции GetDistanceKm
SELECT round(FlightManagementPackage.GetDistanceKm('SVO', 'VVO'),3) AS distance
FROM dual;


SET SERVEROUTPUT ON;

--пример использования процедуры AddNewFlight
BEGIN
    FlightManagementPackage.AddNewFlight(
        p_flight_no             => 'PG7',
        p_scheduled_departure   => TO_DATE('15.09.17 10:00:00', 'DD.MM.RR HH24:MI:SS'),
        p_scheduled_arrival     => TO_DATE('15.09.17 18:00:00', 'DD.MM.RR HH24:MI:SS'),
        p_departure_airport     => 'SVO',
        p_arrival_airport       => 'LED',
        p_status                => 'Scheduled',
        p_aircraft_code         => 'SU9'
    );
END;
/
select * from my_flights;
delete from my_flights where flight_no='PG7';


--пример использования доп функции которая считает выручку реса за определённый
диапазон дат
SELECT FlightManagementPackage.MyFunction(
         'PG0134',
         TO_DATE('10.07.17 00:00:00', 'DD.MM.RR HH24:MI:SS'),
         TO_DATE('28.07.17 23:59:59', 'DD.MM.RR HH24:MI:SS')
       ) as revenue
FROM dual;
```

15 FlightManagementPackage

```
--создаём временную таблицу чтобы не затереть исходные
create table my_flights as select * from avia.flights;
```

```
--функция создания инкремента
DECLARE
  v_new_start NUMBER;
BEGIN

  SELECT NVL(MAX(flight_id), 0) + 1 INTO v_new_start FROM my_flights;


  BEGIN
    EXECUTE IMMEDIATE 'DROP SEQUENCE flights_seq';
  EXCEPTION
    WHEN OTHERS THEN
      NULL;
  END;


  EXECUTE IMMEDIATE 'CREATE SEQUENCE flights_seq START WITH ' || v_new_start ||
                  ' INCREMENT BY 1 NOCACHE NOCYCLE';

  DBMS_OUTPUT.PUT_LINE('Создан инкремент flights_seq, начальное значение ' ||
v_new_start);
END;
/

set serveroutput on;

--создание пакета FlightManagementPackage
CREATE OR REPLACE PACKAGE FlightManagementPackage AUTHID DEFINER AS
    TYPE flight_refcursor IS REF CURSOR;

    FUNCTION GetAvailableFlights(
        start_date DATE,
        end_date DATE,
        departure_airport_code VARCHAR2
    ) RETURN flight_refcursor;

    FUNCTION GetDistanceKm(
        airport_code1 VARCHAR2,
        airport_code2 VARCHAR2
    ) RETURN NUMBER;

    PROCEDURE AddNewFlight(
        p_flight_no             IN avia.flights.flight_no%TYPE,
        p_scheduled_departure   IN avia.flights.scheduled_departure%TYPE,
        p_scheduled_arrival     IN avia.flights.scheduled_arrival%TYPE,
        p_departure_airport     IN avia.flights.departure_airport%TYPE,
        p_arrival_airport       IN avia.flights.arrival_airport%TYPE,
```

```
    p_status              IN avia.flights.status%TYPE,
    p_aircraft_code       IN avia.flights.aircraft_code%TYPE
  );

  FUNCTION MyFunction(
    p_flight_no   IN VARCHAR2,
    p_start_date  IN DATE,
    p_end_date    IN DATE
  ) RETURN VARCHAR2;

  FUNCTION GetTopSpenderByCity(
    p_airport_code IN VARCHAR2,
    p_start_date   IN DATE,
    p_end_date     IN DATE
  ) RETURN VARCHAR2;


END FlightManagementPackage;
/


--создание тела пакета FlightManagementPackage
CREATE OR REPLACE PACKAGE BODY FlightManagementPackage AS

  FUNCTION GetAvailableFlights(
    start_date DATE,
    end_date DATE,
    departure_airport_code VARCHAR2
  ) RETURN flight_refcursor IS
    flights flight_refcursor;
  BEGIN
    OPEN flights FOR
      SELECT f.flight_id,
             f.flight_no,
             f.scheduled_departure,
             f.departure_airport,
             f.status
      FROM avia.flights f
      WHERE f.departure_airport = departure_airport_code
        AND f.scheduled_departure >= start_date
        AND f.scheduled_departure < end_date + 1
        AND f.status IN ('Scheduled')
        AND EXISTS (
              SELECT 1
              FROM avia.seats s
              JOIN avia.flights f2 ON s.aircraft_code = f2.aircraft_code
              WHERE f2.flight_id = f.flight_id
                AND s.seat_no NOT IN (
```

```
                    SELECT seat_no
                    FROM avia.boarding_passes
                    WHERE flight_id = f.flight_id
                )
            );
        RETURN flights;
    END GetAvailableFlights;

    FUNCTION GetDistanceKm(
        airport_code1 VARCHAR2,
        airport_code2 VARCHAR2
    ) RETURN NUMBER IS
        lon1 NUMBER;
        lat1 NUMBER;
        lon2 NUMBER;
        lat2 NUMBER;
        dist NUMBER;
        pi CONSTANT NUMBER := ACOS(-1);
    BEGIN
        EXECUTE IMMEDIATE 'ALTER SESSION SET NLS_NUMERIC_CHARACTERS =
'.,'';
        SELECT
            TO_NUMBER(REGEXP_SUBSTR(coordinates, '[-0-9.]+', 1, 1)),
            TO_NUMBER(REGEXP_SUBSTR(coordinates, '[-0-9.]+', 1, 2))
        INTO lon1, lat1
        FROM avia.airports_data
        WHERE airport_code = airport_code1;

        SELECT
            TO_NUMBER(REGEXP_SUBSTR(coordinates, '[-0-9.]+', 1, 1)),
            TO_NUMBER(REGEXP_SUBSTR(coordinates, '[-0-9.]+', 1, 2))
        INTO lon2, lat2
        FROM avia.airports_data
        WHERE airport_code = airport_code2;

        dist := 6371 * ACOS(
            COS(lat1 * pi / 180) * COS(lat2 * pi / 180) *
            COS((lon2 - lon1) * pi / 180) +
            SIN(lat1 * pi / 180) * SIN(lat2 * pi / 180)
        );
        RETURN dist;
    END GetDistanceKm;

    PROCEDURE AddNewFlight(
        p_flight_no             IN avia.flights.flight_no%TYPE,
        p_scheduled_departure   IN avia.flights.scheduled_departure%TYPE,
        p_scheduled_arrival     IN avia.flights.scheduled_arrival%TYPE,
        p_departure_airport     IN avia.flights.departure_airport%TYPE,
```

```
        p_arrival_airport       IN avia.flights.arrival_airport%TYPE,
        p_status                IN avia.flights.status%TYPE,
        p_aircraft_code         IN avia.flights.aircraft_code%TYPE
    ) IS
        v_flight_count NUMBER;
        v_max_range NUMBER;
        v_distance NUMBER;
        v_new_flight_id NUMBER;
        v_flight_no my_flights.flight_no%TYPE;

CURSOR cur_flight_exists IS
    SELECT flight_no
    FROM my_flights
    WHERE flight_no = p_flight_no
      AND scheduled_departure = p_scheduled_departure
      AND departure_airport = p_departure_airport
      AND arrival_airport = p_arrival_airport;

BEGIN
    v_flight_count := 0;
    OPEN cur_flight_exists;
    LOOP
        FETCH cur_flight_exists INTO v_flight_no;
        EXIT WHEN cur_flight_exists%NOTFOUND;
        v_flight_count := v_flight_count + 1;
    END LOOP;
    CLOSE cur_flight_exists;

    IF v_flight_count > 0 THEN
        DBMS_OUTPUT.PUT_LINE('Ошибка: Рейс уже существует.');
        RETURN;
    END IF;
        SELECT "RANGE"
          INTO v_max_range
          FROM avia.aircrafts_data
         WHERE aircraft_code = p_aircraft_code;

        v_distance := FlightManagementPackage.GetDistanceKm(p_departure_airport,
p_arrival_airport);

        IF v_max_range < 1.2 * v_distance THEN
            DBMS_OUTPUT.PUT_LINE('Ошибка: Дальность полёта самолёта
недостаточна.');
            RETURN;
        END IF;

        INSERT INTO my_flights (
            flight_id,
```

```
        flight_no,
        scheduled_departure,
        scheduled_arrival,
        departure_airport,
        arrival_airport,
        status,
        aircraft_code
    ) VALUES (
        flights_seq.NEXTVAL,
        p_flight_no,
        p_scheduled_departure,
        p_scheduled_arrival,
        p_departure_airport,
        p_arrival_airport,
        p_status,
        p_aircraft_code
    )
    RETURNING flight_id INTO v_new_flight_id;

    DBMS_OUTPUT.PUT_LINE('Рейс успешно добавлен. Flight ID: ' || v_new_flight_id);

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Ошибка: ' || SQLERRM);
        RETURN;
END AddNewFlight;

FUNCTION MyFunction(
    p_flight_no   IN VARCHAR2,
    p_start_date  IN DATE,
    p_end_date    IN DATE
) RETURN VARCHAR2 IS
    v_revenue NUMBER;
v_dep_airport avia.flights.departure_airport%TYPE;
v_arr_airport avia.flights.arrival_airport%TYPE;
v_result      VARCHAR2(256);
BEGIN
    SELECT NVL(SUM(tf.amount), 0)
      INTO v_revenue
      FROM avia.ticket_flights tf
      JOIN avia.flights f ON tf.flight_id = f.flight_id
      JOIN avia.tickets t ON tf.ticket_no = t.ticket_no
      JOIN avia.bookings b ON t.book_ref = b.book_ref
     WHERE f.flight_no = p_flight_no
       AND b.book_date BETWEEN p_start_date AND p_end_date;

SELECT departure_airport, arrival_airport
  INTO v_dep_airport, v_arr_airport
```

```
      FROM avia.flights
      WHERE flight_no = p_flight_no
        AND ROWNUM = 1;
          v_result :=   TO_CHAR(v_revenue) ||','||p_flight_no  ||','|| v_dep_airport ||','||
v_arr_airport;
        RETURN v_result;
    EXCEPTION
        WHEN OTHERS THEN
          RETURN 0;
    END MyFunction;



    FUNCTION GetTopSpenderByCity(
        p_airport_code IN VARCHAR2,
        p_start_date   IN DATE,
        p_end_date     IN DATE
    ) RETURN VARCHAR2 IS
        v_top_spender VARCHAR2(128);
        v_count       NUMBER;
    BEGIN
        SELECT COUNT(*)
          INTO v_count
          FROM avia.airports_data
         WHERE airport_code = p_airport_code;
        IF v_count = 0 THEN
            RAISE_APPLICATION_ERROR(
              -20030,
              'Неизвестный код аэропорта: ' || p_airport_code
            );
        END IF;

        BEGIN
          SELECT passenger_name
            INTO v_top_spender
          FROM (
            SELECT
              t.passenger_name,
              SUM(tf.amount) AS total_spent
            FROM avia.tickets       t
            JOIN avia.ticket_flights tf ON t.ticket_no = tf.ticket_no
            JOIN avia.flights        f  ON tf.flight_id    = f.flight_id
            WHERE f.departure_airport = p_airport_code
              AND f.scheduled_departure BETWEEN p_start_date AND p_end_date
            GROUP BY t.passenger_name
            ORDER BY total_spent DESC
          ) top_spenders
          WHERE ROWNUM = 1;
```

```
        EXCEPTION
          WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(
              -20031,
              'Нет данных о тратах из аэропорта ' || p_airport_code || ' за период ' ||
TO_CHAR(p_start_date,'DD.MM.YYYY') || '–' || TO_CHAR(p_end_date,'DD.MM.YYYY')
            );
        END;

        RETURN v_top_spender;
    END GetTopSpenderByCity;



END FlightManagementPackage;
/

--пример использования функции GetAvailableFlights
VAR flights REFCURSOR;

BEGIN
    :flights := FlightManagementPackage.GetAvailableFlights(
        TO_DATE('2017-09-05', 'YYYY-MM-DD'),
        TO_DATE('2017-09-09', 'YYYY-MM-DD'),
        'SVO'
    );
END;
/
PRINT flights;


--пример использования функции GetDistanceKm
SELECT round(FlightManagementPackage.GetDistanceKm('SVO', 'VVO'),3) AS distance
FROM dual;


SET SERVEROUTPUT ON;

--пример использования процедуры AddNewFlight
BEGIN
    FlightManagementPackage.AddNewFlight(
        p_flight_no             => 'PG7',
        p_scheduled_departure   => TO_DATE('15.09.17 10:00:00', 'DD.MM.RR HH24:MI:SS'),
        p_scheduled_arrival     => TO_DATE('15.09.17 18:00:00', 'DD.MM.RR HH24:MI:SS'),
        p_departure_airport     => 'SVO',
        p_arrival_airport       => 'LED',
        p_status                => 'Scheduled',
        p_aircraft_code         => 'SU9'
```

```
    );
END;
/
select * from my_flights;
delete from my_flights where flight_no='PG7';


--пример использования доп функции которая считает выручку реса за определённый диапазон дат
SELECT FlightManagementPackage.MyFunction(
        'PG0134',
        TO_DATE('10.07.17 00:00:00', 'DD.MM.RR HH24:MI:SS'),
        TO_DATE('28.07.17 23:59:59', 'DD.MM.RR HH24:MI:SS')
      ) as revenue
FROM dual;


SET SERVEROUTPUT ON;
DECLARE
  v_name VARCHAR2(128);
BEGIN
  v_name := GetTopSpenderByCity(
    p_airport_code => 'SVO',
    p_start_date   => TO_DATE('2017-01-01','YYYY-MM-DD'),
    p_end_date     => TO_DATE('2025-04-30','YYYY-MM-DD')
  );
  DBMS_OUTPUT.PUT_LINE(CHR(10)||v_name || ' больше всех потратил(-а).');
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE(CHR(10)||'Ошибка: ' || SQLERRM);
END;
/
```